

CRAFT: COMPOSITIONAL REWARD ALIGNMENT FOR TARGET-HIDDEN LOOP VERIFICATION

Anonymous authors

Paper under double-blind review

ABSTRACT

Valid loop invariants are closed under conjunction: useful clauses from different responses can prove a target even when no response does so alone. Yet standard training scores each response as an indivisible answer. We study this mismatch under *target-hidden loop verification*, where target assertions and postconditions $\mathcal{G}$ are withheld during generation and training. CRAFT aligns both stages through rollout-decompose-pool-filter-compose. At inference, it decomposes responses into clauses, pools them across responses, filters the pool with syntax checking and Houdini, and composes the survivors into one invariant before restoring $\mathcal{G}$. Training uses the same object: rejection-filtered SFT distills accepted multi-response pools, while RL scores each response by the negative coverage of its pooled survivors and uses Shapley allocation to preserve complementary exploration. The only learned generator in both stages is the small open-weight model being trained: SFT acceptance and RL rewards use only program executions and verifier outcomes, without an external teacher or reward model. On 832 C programs, the inference pipeline raises bare Qwen3-8B from 6.43% pass@1 to 37.86% compose@1. Large-scale SFT then reaches 65.50% compose@1. A subsequent RL stage uses only 4,992 records—15.0% as many as SFT, with partial source overlap—yet raises compose@1 to 69.23%. This 3.73-point gain solves 31 additional programs and reduces verification error by 10.8%. The gain is concentrated at the primary metric, compose@1: compose@4–32 stays within one point of the SFT checkpoint. RL therefore amortizes the compositional capability learned by SFT into the first rollout. Under the same target-hidden reasoning-off protocol, one RFT+RL Qwen3-8B rollout exceeds Claude Sonnet-4.6 at compose@1 by 12.26 points. Reward ablations show why this compression preserves exploration: Shapley credit adds 9.85–11.30 compose points over clause-decomposed credit, while binary and whole-response rewards collapse composition.

1 INTRODUCTION

Loop invariants turn unbounded executions into finite proofs, but validity alone does not imply usefulness: *true* is valid for every loop and proves nothing downstream. At the same time, valid invariants are closed under conjunction. A range bound from one response and a relational equality from another can therefore prove a target that neither response proves alone, while clause-wise verification can discard failures without losing useful clauses. This makes the filtered cross-response clause pool, rather than one complete response, the natural inference object.

Whole-response rewards are mismatched to this object. They erase credit for a useful clause when a sibling clause fails, and they value a complementary constraint no more than one repeated by every peer. More generally, response-level RLVR can sharpen probability on already-supported answers without expanding large- k coverage (Yue et al., 2025), potentially reducing the diversity that composition needs. Our central claim is therefore an alignment claim: the training reward should value the same verified composition that inference constructs from a fixed rollout group, rather than each rollout as an independent final answer.

We study this claim under *target-hidden loop verification*. The model sees the loop context but not the target assertions and postconditions $\mathcal{G}$. After the invariant set is fixed, the untouched source is restored and the verifier checks both induction and every original target. Hiding $\mathcal{G}$ prevents target copying

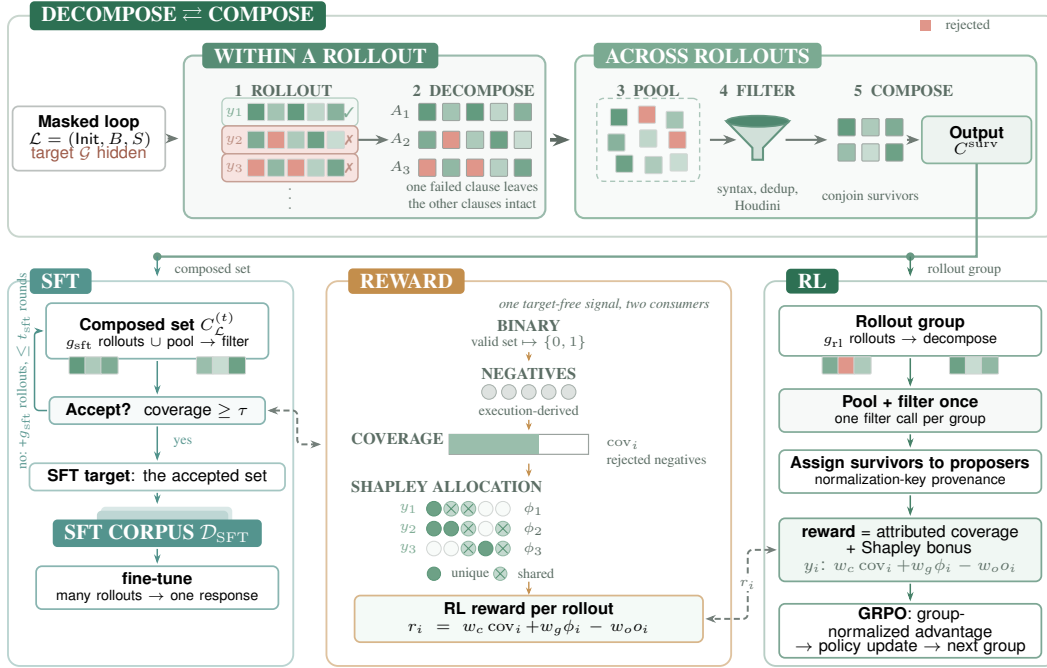


Figure 1: **Decompose-compose is the organizing principle.** Target-hidden rollouts are decomposed into clauses, pooled, filtered, and composed into one invariant (top). SFT distills an accepted multi-response pool into one response (left). The shared scorer uses execution-derived negative coverage and Shapley allocation (center); RL assigns pooled survivors to their proposers and optimizes the resulting rewards (right). The hidden target $\mathcal{G}$ is restored only for final verification.

and target-specific proof feedback, so generation and training require a graded, target-independent strength signal.

At inference, CRAFT implements *rollout-decompose-pool-filter-compose*: given k responses, it extracts and pools clauses, filters them with syntax checks and Houdini, and then restores $\mathcal{G}$. $\text{compose}@k$ measures the verified rate of this composed prefix, preserving partial answers that $\text{pass}@k$ discards; $\text{compose}@1$ is our primary optimization and reporting metric. Training is aligned to this object at the same two compositional levels. Within a rollout, execution-derived negative coverage rewards each response according to the clauses retained by the pooled inference filter; across each GRPO group (Shao et al., 2024), a closed-form Shapley term discounts coverage repeated by multiple peers. SFT distills the same filtered multi-response union into one response. The two training stages therefore have distinct roles: SFT raises the compositional ceiling, RL amortizes that capability into the first rollout, and Shapley credit enables this compression without collapsing complementary exploration. Both stages are self-generated: the only learned generator is the small open-weight model being trained, while the program executor and verifier provide every SFT acceptance decision and RL reward. No external teacher model generates targets, critiques responses, or supplies reward.

Contributions. This paper makes three contributions:

- We introduce a rollout–decompose–pool–filter–compose inference framework for target-hidden invariant generation and use compose@1 as its primary metric. On Qwen3-8B, the framework raises one-response verification from 6.43% pass@1 to 37.86% compose@1, a 31.43-point gain.
- We introduce a two-stage training pipeline: rejection-filtered SFT raises Qwen3-8B compose@1 to 65.50%, and a subsequent 4,992-record compositional RL stage reaches 69.23%. RL adds 3.73 points, reduces verification error by 10.8%, and produces an 8B model that exceeds Claude Sonnet-4.6 at compose@1 by 12.26 points. Target synthesis and reward computation require only the trained open-weight model, program execution, and verification, with no external teacher model.
- Through controlled reward and sampler ablations, cross-model evaluation, and 832-program analysis, we show that decompose–compose RLVR provides effective credit for invariant generation. Clause-decomposed feedback preserves partially correct responses, while Shapley allocation adds 9.85–11.30 compose points over clause-only credit and mitigates the exploration collapse of binary and whole-response rewards.

2 PRELIMINARIES

Loop verification. A task contains a loop $\mathcal{L} = (\text{Init}, B, S)$ and targets $\mathcal{G} = \{(\ell, g_\ell)\}$, where Init describes loop entry, B is the guard, S is the body, and g_ℓ occurs at program location ℓ . In this paper, *valid invariant* means inductive:

$$\text{Init} \Rightarrow I, \quad \{I \wedge B\} S \{I\},$$

which implies every reachable loop-head state satisfies I (Floyd, 1967; Hoare, 1969). A finite clause set C composes as $I(C) = \bigwedge_{c \in C} c$ and is *jointly inductive* when $I(C)$ is valid. It is target-sufficient when it proves every g_ℓ at its original location; for a post-loop target this reduces to $I(C) \wedge \neg B \Rightarrow g$.

Target-hidden verification. Generation and intermediate scores use $\mathcal{L}$, not $\mathcal{G}$. Targets and target-revealing text are masked, while preconditions and executable loop semantics remain visible. The invariant is fixed before the original targets are restored for final verification. We study scalar-integer C programs with one loop; Appendix C gives the scope and an example.

3 RELATED WORK

Invariant generation and verified filtering. Classical invariant synthesis uses abstract interpretation, affine relations, templates, interpolation, syntax-guided search, or counterexamples (Cousot & Cousot, 1977; Karr, 1976; McMillan, 2003; Alur et al., 2013; Garg et al., 2014); Houdini retains an inductive subset of proposed clauses (Flanagan & Leino, 2001). Neural and LLM systems learn construction policies or combine generation with solver feedback, static analysis, filtering, and repair (Si et al., 2018; 2020; Ryan et al., 2020; Yao et al., 2020; Yu et al., 2023; Kamath et al., 2023; Chakraborty et al., 2023; Cao et al., 2025; Wu et al., 2024a; Liu et al., 2024b; Wen et al., 2024; Ma et al., 2025; Yang et al., 2026b; Wu et al., 2024b; First et al., 2023). CRAFT instead makes the filtered union of multiple responses the shared inference and training object.

Training from formal and behavioral feedback. Verified data and execution or prover feedback have been used to train invariant, code, and proof models (Wei et al., 2025; Pinto et al., 2026; Le et al., 2022; Yan et al., 2025). COINS and SpecRL further use positive and negative behaviors to distinguish weak valid specifications from stronger ones (Yang et al., 2026a; Huang et al., 2026). Our signal is likewise behavioral but is derived from target-hidden loop executions and applied only after pooled clauses survive verification.

Set-level credit in verifiable-reward RL. Per-response RL can improve pass@1 without expanding large- k coverage (Yue et al., 2025; He et al., 2025). Recent objectives directly optimize pass@ k or up-weight rare correct responses (Walder & Karkhanis, 2025; Chen et al., 2025; He et al., 2025), but still treat each response as an indivisible answer. CRAFT instead credits verified clauses within imperfect responses and allocates group credit by their contribution to pooled negative coverage.

4 METHOD

CRAFT aligns training with compositional inference. At inference, it splits responses into clauses, pools clauses across responses, verifier-filters the pool, and conjoins the survivors. SFT distills this pooled result into one response, while RL rewards each response based on the negative coverage of its pooled survivors and their group-level complementarity. Assertions and postconditions remain hidden throughout generation and training and are restored only for final verification. Appendix A gives the complete formal definition.

4.1 TARGET-HIDDEN COMPOSABLE INFERENCE

For a masked loop $\mathcal{L}$, let $\text{parts}(y)$ be the ordered invariant clauses extracted from response y . Let $\text{Admit}(y) = \text{Unique}(\text{First}_{m_{\max}}(\text{parts}(y)))$, where First_m keeps the first m clauses. The value of $m_{\max}$ is reported in the parameter table rather than embedded in the formula. Algorithm 1 defines the complete inference procedure.

Algorithm 1 Rollout–decompose–pool–filter–compose

Input: task $\mathcal{T} = (\mathcal{L}, \mathcal{G})$, policy π , number of responses n
Output: composed invariant I , final verdict v

- 1: $x \leftarrow \mathcal{L}$ by masking $\mathcal{G}$ in $\mathcal{T} \triangleright$ *hide target*
- 2: $y_{1:n} \sim \pi(\cdot \mid x) \triangleright$ *rollout*
- 3: $A_i \leftarrow \text{Admit}(y_i)$ for $i = 1, \dots, n \triangleright$ *decompose and cap*
- 4: $P \leftarrow \text{Unique}(A_1 \parallel \dots \parallel A_n) \triangleright$ *pool in response order*
- 5: $C \leftarrow \text{Filter}_{\mathcal{L}}(P) \triangleright$ *filter*
- 6: $I \leftarrow \bigwedge_{c \in C} c \triangleright$ *compose*
- 7: $v \leftarrow \text{TaskVerify}(\mathcal{T}, C) \triangleright$ *restore target*
- 8: **return** (I, v)

Unique performs conservative duplicate removal, and Filter is resource-bounded Houdini: establishment and preservation are checked clause by clause under the current candidate family, so clauses from different responses may support one another. Timeout and unknown cause deletion; a set is returned only after all its clauses are proved in their final joint context, while hitting the round limit returns the conservative empty result. Appendix A.1 separates the semantic definition of an invariant from the Frama-C-aligned loop-verifier interface; Appendix A.2 defines both ideal and bounded Houdini. All clause statuses in one round may use the pre-deletion candidate context, including hypotheses whose own goals fail. Houdini therefore reruns the survivors after every deletion and stops only when one complete round deletes nothing.

Only after I is fixed do we restore the hidden targets $\mathcal{G}$ and run the final verification conditions at their original locations. The inference objective is therefore

$$\max_{\pi} \Pr_{y_{1:n} \sim \pi(\cdot \mid \mathcal{L})} [\text{TaskVerify}((\mathcal{L}, \mathcal{G}), C(y_{1:n}))].$$

Its empirical estimate is $\text{compose}@n$. It can exceed $\text{pass}@n$ because a conjunction of valid invariants remains valid and may combine complementary bounds and relations. Appendix A gives the complete definitions and algorithms; Appendix B states and proves the composition and filtering properties.

4.2 TARGET-INDEPENDENT STRENGTH SIGNAL

Because targets are hidden, training cannot score direct target proofs. We instead execute the masked loop to collect reachable loop-head states $\mathcal{E}$ and construct target-independent negative traces $\mathcal{N}$. The traces are either local relational perturbations or continuations beyond a witnessed loop exit. Conservative cleaning removes observed reachable states, possible fresh entries, duplicates, and unsupported states; Appendix D.2 gives the construction.

For a verified clause set C , let $\text{rej}(C)$ contain every negative trace on which some clause is false. Its trajectory-level coverage is

$$\text{cov}(C) = \frac{|\text{rej}(C)|}{|\mathcal{N}|}, \quad |\mathcal{N}| > 0. \quad (1)$$

Each trace contributes one unit regardless of its length. Instances with no retained negatives are not used for SFT acceptance; RL uses the binary all-admitted-clauses-survive fallback defined in Algorithm 4.

4.3 DISTILLING MULTI-RESPONSE SEARCH

SFT uses Algorithm 1 as a target synthesizer. Starting from an empty accumulated clause pool, each round adds a new group sampled from the same small open-weight base model that is subsequently fine-tuned. Executor-derived traces and verifier outcomes jointly determine which candidate sets become SFT targets; no external model proposes labels or judges responses. Each round adds the new base-model clauses and re-filters the entire accumulated pool:

$$P_t = \text{Unique}\left(P_{t-1} \parallel \text{Admit}(y_1^{(t)}) \parallel \dots \parallel \text{Admit}(y_{g_{\text{sft}}}^{(t)})\right), \quad C_t = \text{Filter}_{\mathcal{L}}(P_t). \quad (2)$$

Re-filtering the accumulated clause pool allows a previously removed clause to return when a later response supplies a supporting clause. The first set satisfying $\text{cov}(C_t) \geq \tau$ is serialized as one SFT target, where τ is the acceptance threshold; otherwise the instance is dropped after the synthesis budget is exhausted. Neither the SFT input nor its target contains $\mathcal{G}$.

4.4 CLAUSE AND GROUP CREDIT

RL applies exactly the same pool-then-filter order once per response group. The current small open-weight policy produces the group, and its rewards are computed entirely from executor-derived traces, verifier-retained clauses, and deterministic provenance. No external LLM, critic model, or target assertion contributes to the reward. Let $L_i = \text{parts}(y_i)$, $o_i = \max(0, |L_i| - m_{\text{max}})$, and $A_i = \text{Admit}(y_i)$. Thus $\text{First}_m(L)$ is the prefix containing at most m clauses, o_i counts clauses beyond the per-response cap, while A_i retains the admitted clauses and their provenance. The clauses are then de-duplicated globally and filtered only after the full group has been pooled:

$$P^* = \text{Unique}(A_1 \parallel \dots \parallel A_{g_{\text{rl}}}), \quad C^* = \text{Filter}_{\mathcal{L}}(P^*), \quad (3)$$

$$V_i = \{c \in A_i : \text{key}(c) \in \text{keys}(C^*)\}, \quad R_i = \text{rej}(V_i). \quad (4)$$

Here $\text{keys}(C^*)$ is the family of canonical keys of the pooled survivors. Thus V_i assigns each pooled survivor back to every response that proposed it. A response retains the coverage contributed by its surviving clauses even when its other clauses fail, while mutually supporting clauses are checked in the same pooled context used at inference. The cheap reachable-state check is an internal front-end of this single pooled filtering call; it does not filter responses separately.

Coverage alone gives the same score to a unique discovery and a clause already supplied by every peer. Let $f(\nu) = |\{j : \nu \in R_j\}|$. We add the closed-form Shapley allocation of the fixed coverage game:

$$\phi_i = \frac{1}{|\mathcal{N}|} \sum_{\nu \in R_i} \frac{1}{f(\nu)}. \quad (5)$$

The rollout reward is

$$r_i = w_c \frac{|R_i|}{|\mathcal{N}|} + w_g \phi_i - w_o o_i. \quad (6)$$

Here w_c , w_g , and w_o denote the coverage, group-credit, and overflow weights. The first term rewards each response by the negative coverage of its pooled survivors; the second is a complementarity bonus that divides redundant coverage among its proposers; the final term penalizes clauses emitted beyond the response cap. It is the Shapley value after filtering the full group, not the value of a game that reruns Houdini for every coalition. A clause that only supports another clause receives no direct coverage credit.

GRPO normalizes these rewards within the sampled group and applies the standard clipped policy update (Appendix A.7). Training and inference are aligned because inference, SFT synthesis, and RL scoring all use the same operation—filtering the union of decomposed responses—while the reward remains independent of the hidden target.

5 EXPERIMENTS

We ask how composition scales with sampling (RQ1), how much CRAFT improves $\text{compose}@k$ over standalone responses (RQ2), how CRAFT compares with specialized tools (RQ3), how much RL adds (RQ4), and what drives the training gain (RQ5).

5.1 EXPERIMENTAL SETUP

Data and judge. Training data comprise 4,992 RL records and 33,346 SFT records (31,865 distinct sources) of target-masked scalar-integer single-loop programs; Appendix H gives their synthesis and verification. SFT targets and RL rewards are produced only by the open-weight model being trained, program execution, and Frama-C/WP verification; no external teacher or closed-model supervision is used. The evaluation combines 832 programs from Code2Inv, SyGuS, SV-COMP, LIPuS, Clause2Inv, and Loopy: 316 linear, 50 nonlinear-arithmetic, and 466 Loopy instances (Si et al., 2020; Alur et al., 2013; Beyer, 2023; Yu et al., 2023; Cao et al., 2025; Kamath et al., 2023). No training program matches an evaluation program under exact, alpha-renamed, or constant-abstracted matching (Appendix G). The judge is Frama-C 27.1 (Cobalt) with the WP plug-in, Typed memory model, and Alt-Ergo/Z3 prover portfolio (Kirchner et al., 2015). Success requires it to prove induction and every restored target; all other outcomes count as zero.

Inference protocol. All CRAFT evaluation runs share target masking, the canonical prompt, clause processing, Houdini, stochastic decoding, and no re-roll or target feedback; external tools retain their own target-hidden orchestration. RQ1 reuses one saved pool of n_{probe} responses per task. Appendix I gives decoding, serving, prover, and training details.

Metrics. Throughout evaluation, success means $\mathcal{G}$ -sufficiency under the final verifier. $\text{pass}@k$ estimates whether any of k responses succeeds alone; $\text{compose}@k$ measures whether filtering their clause union succeeds. The former uses the standard unbiased estimator, while the latter evaluates saved response prefixes; Appendix I gives both estimators. We also report k_α , the smallest $k \in \mathcal{K}$ whose $\text{compose}@k$ reaches the saturation fraction α of $\text{compose}@k_{\text{max}}$, where $k_{\text{max}} = \max \mathcal{K}$. A smaller k_α means the retained pool reaches its large-budget performance sooner.

5.2 RQ1: HOW DO PASS@K AND COMPOSE@K SCALE?

We sample n_{probe} target-hidden responses per task from six base checkpoints: Qwen3-1.7B, Qwen3-4B, Qwen3-8B, Qwen3-14B, Qwen3-30B-A3B (Yang et al., 2025), and Llama 3.1 8B. Complete curves, k_α , and whole-workload values appear in Appendix 9.

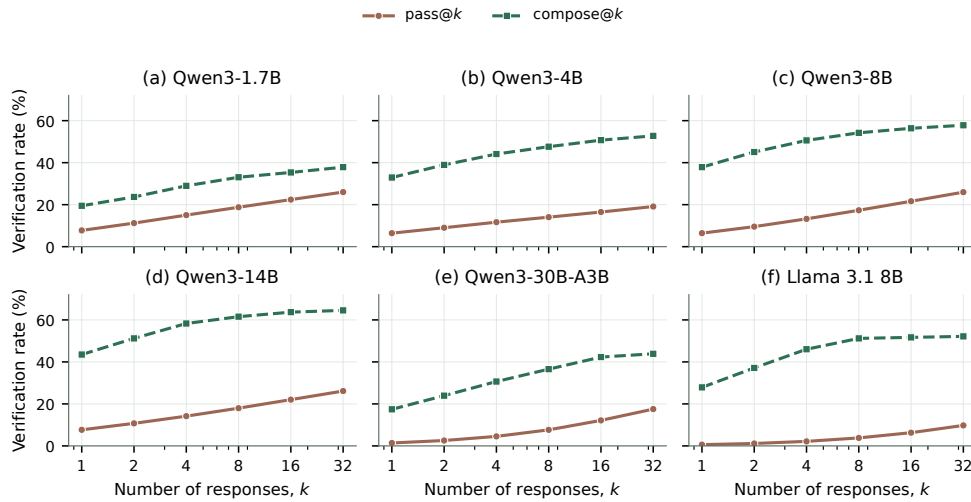


Figure 2: $\text{pass}@k$ (unbiased estimate) and prefix $\text{compose}@k$. Composition saturates by $k \approx 8-16$, while $\text{pass}@k$ keeps growing.

Insight. Standalone completion and filtered composition scale differently. Across all six checkpoints, $\text{compose}@16$ is within 2.6 points of $\text{compose}@32$, whereas $\text{pass}@k$ continues improving through $k=32$. Llama 3.1 8B shows the same pattern despite low standalone accuracy ($\text{pass}@32$ 9.8%): composition rises from 27.9% at $k=1$ to 51.7% at $k=16$. Thus the first dozen samples contain most reusable clauses, while complete standalone proofs benefit from wider sampling. Performance is not monotone in nominal model size: Qwen3-30B-A3B reaches 43.87% $\text{compose}@32$, below the 8B and 14B dense checkpoints. RQ2 gives the matched pipeline comparison.

5.3 RQ2: HOW MUCH DOES CRAFT IMPROVE $\text{COMPOSE}@k$ OVER STANDALONE RESPONSES?

We evaluate the same saved response pools at $k=1, 8$. Framework-free rows use the unbiased $\text{pass}@k$ estimator over each pool, while pipeline rows apply prefix $\text{compose}@k$ to the first k saved responses. Zero is the untrained checkpoint; the trained rows separate SFT from inference-time composition. RL is evaluated separately in Section 5.5.

Checkpoint	$\text{pass}@1$	$\text{compose}@1$	$\text{pass}@8$	$\text{compose}@8$
<i>Qwen3-4B</i>				
Zero	6.42	32.93	14.05	47.60
SFT	40.43	62.62	59.99	76.20
<i>Qwen3-8B</i>				
Zero	6.43	37.86	17.33	54.21
SFT	41.93	65.50	61.11	77.52
<i>Qwen3-14B</i>				
Zero	7.67	43.51	17.97	61.54
SFT	45.66	66.35	63.50	78.49
<i>Llama 3.1 8B</i>				
Zero	0.61	27.88	3.81	51.20

Table 1: Whole-workload comparison across models and training stages (% verified, non-reasoning decoding). $\text{pass}@k$ and $\text{compose}@k$ use the estimators defined in the experimental setup. SFT results are available for the three dense Qwen3 checkpoints. Complete base-checkpoint curves appear in Appendix 9; complete SFT curves and per-stratum results appear in Appendix 11. API models under the same protocol appear in Table 16 (Appendix J.6).

Insight. On Qwen3-8B at $k=1$, the inference pipeline raises verification from 6.43% $\text{pass}@1$ to 37.86% $\text{compose}@1$: a 31.43-point absolute and 488.8% relative gain from decomposing and filtering the same response. Across the four base checkpoints in Table 1, the inference pipeline adds 26.5–35.8 points at $k=1$ and 33.6–47.4 points at $k=8$ under fixed response-generation pools, without additional model calls. Qwen3-8B SFT further raises $\text{compose}@1$ to 65.50%, adding 27.64 points or 73.0% over the untrained pipeline. End to end, this is a 59.07-point (918.7%) gain over bare $\text{pass}@1$ and exceeds the untrained Qwen3-14B pipeline by 21.99 points. After SFT, the three Qwen checkpoints reach 62.62–66.35% $\text{compose}@1$ and 76.20–78.49% $\text{compose}@8$ on all 832 tasks; Qwen3-14B is strongest at both budgets.

5.4 RQ3: HOW DOES CRAFT COMPARE WITH SPECIALIZED TOOLS?

We compare CRAFT with the target-hidden pipelines of AutoSpec, SESpec, Clause2Inv, Loopy, and Daikon (Wen et al., 2024; Yang et al., 2026b; Cao et al., 2025; Kamath et al., 2023; Ernst et al., 2007). All LLM-based systems use GPT-5-nano, and all outputs are judged on the same untouched programs by Frama-C/WP. Each system retains its native, predeclared orchestration, so model-call and token budgets differ; Table 2 reports both token and runtime costs. Unsupported inputs count as failures in the common 832-program denominator.

The Naive baseline uses one call without invariant-specific instructions; CRAFT instead elicits a clause pool and filters it before composition.

System	Linear / 316	NLA / 50	Loopy / 466	All / 832	Tokens / task	Time / task
AutoSpec	47.78%	6.00%	42.27%	42.19%	2,181.72	27.34 s
SESpec	28.80%	4.00%	33.05%	29.69%	22,081.61	68.50 s
Clause2Inv	20.89%	0.00%	0.00%	7.93%	1,056.25	8.41 s
Daikon	23.10%	0.00%	21.67%	20.91%	0	4.89 s
Loopy	20.25%	0.00%	19.74%	18.75%	14,513.37	147.16 s
Naive	15.19%	2.00%	18.88%	16.47%	225.95	2.98 s
CRAFT (pass@1)	2.53%	0.00%	4.72%	3.61%	1,135.83	7.86 s
CRAFT (compose@1)	34.81%	22.00%	34.55%	33.89%	1,136.82	24.52 s
CRAFT (compose@4)	48.10%	30.00%	51.72%	49.04%	1,905.10	46.40 s
CRAFT (compose@8)	55.38%	44.00%	56.87%	55.53%	2,929.47	69.99 s

Table 2: Common target-hidden comparison under non-reasoning decoding, using the same untouched inputs and Frama-C/WP final judge. All LLM-driven rows share the same fixed backbone model; tokens and time are mean totals per task. The three compose rows are recomputed from one archived ten-rollout pool by prefix-subset composition, unioning the first k responses through the same filter chain; their tokens count one prompt plus k archived completions, and their time adds the measured filter-and-judge time at budget k to the single-request latency.

Insight. With four composed responses, CRAFT verifies 49.0% of the 832 programs, versus 42.2% for the strongest external baseline; eight responses reach 55.5%. The gain also appears on three further API models (Appendix 16). This is an end-to-end comparison: systems retain their native orchestration and therefore differ in cost.

5.5 RQ4: HOW MUCH DOES REINFORCEMENT LEARNING IMPROVE INFERENCE?

We compare matched Qwen3-8B checkpoints before and after RL, holding inference fixed within each pair.

What to expect. Whole-response RL can sharpen probability mass around existing answers (Yue et al., 2025); our clause-level group reward instead predicts higher compose@ k , especially at small budgets, without requiring higher standalone accuracy.

Insight. RL raises Qwen3-8B compose@1 from 37.86% to 57.93%, a 20.07-point absolute and 53.0% relative gain over the untrained pipeline, while preserving pass@1 (6.80% vs. 6.43%). Relative to bare pass@1, the RL path gains 51.50 points (800.9%) and exceeds the untrained Qwen3-14B pipeline by 14.42 points. One RL sample already matches the initialization’s compose@32 (57.81%), a 32-fold budget reduction, and compose@32 rises to 73.20%. This is the predicted redistribution toward complementary clauses rather than better whole responses. Qwen3-14B shows the same compositional shift (Appendix 13), although its pass@1 also rises.

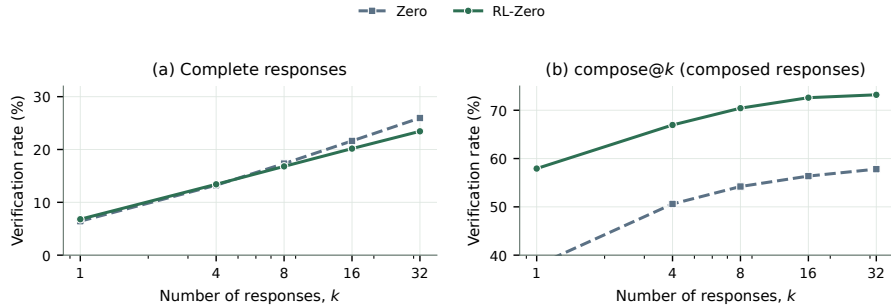


Figure 3: Response-level vs. compositional effects of RL (Qwen3-8B, Zero initialization). RL primarily shifts compose@ k , reaching the initialization’s compose@32 at the *crossing budget* $k^*=1$, the smallest budget whose compose@ k matches the initialization’s compose@32.

5.6 RQ5: WHAT DRIVES THE TRAINING GAIN?

We ablate the reward and negative sampler on Qwen3-8B from the Zero initialization, changing only the component under study. Exact definitions and numbers are in Appendix J.4.

Reward-function ablation. Binary and Whole-rollout give response-level credit; Clause-decomposed pays each response according to the coverage of its surviving subset V_i , and Full adds the group credit $w_g\phi_i$ from Equation (6). All variants process at most $m_{\max}$ clauses per response.

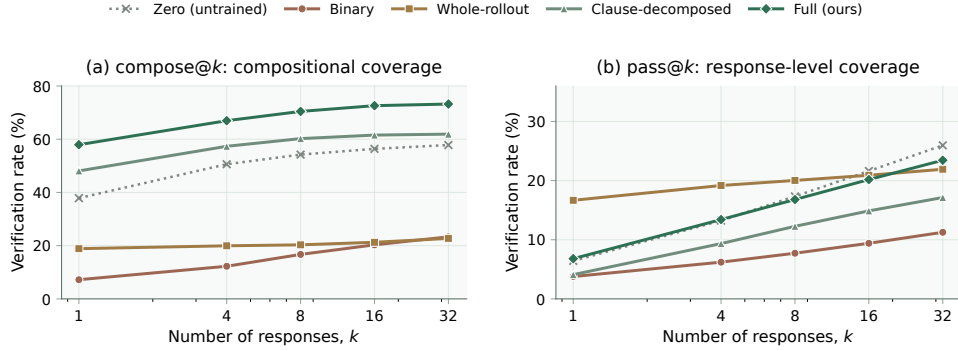


Figure 4: Reward-function ablation on Qwen3-8B (Zero initialization). Whole-rollout credit raises $\text{pass}@1$ (b) but collapses $\text{compose}@k$ (a) far below the untrained model; binary credit collapses both. Clause-decomposed credit surpasses the untrained compose curve, and Full is best at every budget. Dotted: Zero, the untrained checkpoint. Exact numbers in Table 14 (Appendix J.4).

Insight: credit structure controls exploration. Binary and whole-response RLVR treat each rollout as indivisible and fail on the compositional objective. Whole-rollout credit raises $\text{pass}@1$ by 10.24 points, sharpening probability around complete answers, but reduces $\text{compose}@32$ by 35.09 points (60.7% relative); binary credit lowers both metrics. Clause-decomposed credit preserves useful parts of partially correct responses and improves $\text{compose}@1$ and $\text{compose}@32$ over the untrained model by 10.22 and 4.09 points. Group credit then discounts repeated coverage and adds another 9.85–11.30 points, bringing the full reward’s gains to 20.07 points at $\text{compose}@1$ and 15.39 points at $\text{compose}@32$. Matching the credit unit to decompose–compose inference provides denser feedback and mitigates the exploration collapse of whole-response GRPO.

Negative-sampler ablation. Under the full reward, the structured sampler (relation and post-exit families, Section 4.2) exceeds uniform random perturbations by 2.0–5.3 $\text{compose}@k$ points, with little $\text{pass}@k$ change (Table 15).

Does negative coverage predict hidden-target utility? Across 5,606 continuously scored target-hidden candidate sets, negative coverage predicts restored-target success with AUROC 0.738 (clustered 95% CI 0.711–0.763) and AUPRC 0.730. Within-program ranking is stronger (macro AUROC 0.794, permutation $p < 2 \times 10^{-4}$). The association weakens on NLA, so coverage is useful but not universal; Appendix J.5 gives the full protocol.

6 CONCLUSION AND LIMITATIONS

CRAFT aligns training with compositional inference: its pipeline composes rollouts, SFT distills their union, and target-independent RL rewards useful responses through their pooled survivors while discounting repeated coverage. On Qwen3-8B, the pipeline alone raises one-response verification from 6.43% to 37.86% (+31.43 points; +488.8%). Rejection-filtered SFT raises $\text{compose}@1$ to 65.50%, and the subsequent compositional RL stage reaches 69.23%. Together they deliver a 62.80-point (976.7%) end-to-end gain, reduce the SFT checkpoint’s verification error by 10.8%, and exceed Claude Sonnet-4.6 at $\text{compose}@1$ by 12.26 points. Whole-response RL sharpens $\text{pass}@1$ while

collapsing compose@32; decomposed credit preserves partial candidates, and group credit rewards complementary coverage, mitigating the corresponding exploration collapse in GRPO.

The interface extends to reusable atoms that can be pooled and soundly filtered. Specification and test-case generation are potential instances, with task-specific operators, verifiers, and signals; Appendix A states the formal conditions.

Evidence is limited to scalar-integer single-loop C programs, Frama-C/WP, ACSL, and the evaluated checkpoints. Provenance credit is conditioned on the sampled group rather than exact coalition Shapley values; Appendices K and M give the evaluation scope and pooled-credit measurements.

AI USE STATEMENT

Generative AI tools are integral to the evaluated system: they produce the invariant candidates and synthetic SFT data described in the paper. They were also used during research to refine hypotheses, methodology, and experiments; formulate and check mathematical claims; implement and test software; analyze and interpret experimental artifacts; prepare figures and tables; search and summarize literature; and draft, edit, and translate manuscript text. The authors reviewed all AI-assisted work and tested generated code. They take responsibility for independently validating the final content, empirical claims, and released artifacts.

REPRODUCIBILITY STATEMENT

The method and objective are specified in Section 4; Appendices D–I give the implementation defaults, prompt, training interface, data-overlap audit, and evaluation protocol. Appendix J organizes the available measurements.

REFERENCES

- Rajeev Alur, Rastislav Bodík, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. Syntax-guided synthesis. In *Proc. Formal Methods in Computer-Aided Design (FMCAD)*, pp. 1–8, 2013.
- Dirk Beyer. Competition on software verification (SV-COMP). <https://sv-comp.sosy-lab.org/>, 2023.
- Weining Cao, Guangyuan Wu, Tangzhi Xu, Yuan Yao, Hengfeng Wei, Taolue Chen, and Xiaoxing Ma. Clause2Inv: A generate-combine-check framework for loop invariant inference. *Proceedings of the ACM on Software Engineering*, 2(ISSTA):1009–1030, 2025. doi: 10.1145/3728920.
- Saikat Chakraborty, Shuvendu K. Lahiri, Sarah Fakhoury, Madanlal Musuvathi, Akash Lal, Aseem Rastogi, Aditya Senthilnathan, Rahul Sharma, and Nikhil Swamy. Ranking LLM-generated loop invariants for program verification. In *Findings of the Association for Computational Linguistics: EMNLP*, pp. 9164–9175, 2023. doi: 10.18653/v1/2023.findings-emnlp.614.
- Zhipeng Chen, Xiaobo Qin, Youbin Wu, Yue Ling, Qinghao Ye, Wayne Xin Zhao, and Guang Shi. Pass@k training for adaptively balancing exploration and exploitation of large reasoning models. *arXiv preprint arXiv:2508.10751*, 2025.
- Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proc. 4th ACM SIGACT-SIGPLAN Symp. on Principles of Programming Languages (POPL)*, pp. 238–252, 1977.
- Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. The Daikon system for dynamic detection of likely invariants. *Science of Computer Programming*, 69:35–45, 2007.
- Emily First, Markus N. Rabe, Talia Ringer, and Yuriy Brun. Baldur: Whole-proof generation and repair with large language models. In *Proc. 31st ACM Joint European Software Engineering Conf. and Symp. on the Foundations of Software Engineering (ESEC/FSE)*, pp. 1229–1241, 2023.

540 Cormac Flanagan and K. Rustan M. Leino. Houdini, an annotation assistant for ESC/Java. In *Proc.*
541 *Int. Symp. of Formal Methods Europe (FME)*, pp. 500–517. Springer, 2001.

542 Robert W. Floyd. Assigning meanings to programs. In *Proc. Symp. in Applied Mathematics:*
543 *Mathematical Aspects of Computer Science*, volume 19, pp. 19–32, 1967.

544 Pranav Garg, Christof Löding, P. Madhusudan, and Daniel Neider. ICE: A robust framework for
545 learning invariants. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 69–87. Springer,
546 2014.

547 Andre He, Daniel Fried, and Sean Welleck. Rewarding the unlikely: Lifting GRPO beyond dis-
548 tribution sharpening. In *Proceedings of the 2025 Conference on Empirical Methods in Natural*
549 *Language Processing*, 2025.

550 C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12
551 (10):576–580, 1969.

552 Zhechong Huang, Zhao Zhang, Zeyu Sun, Huifeng Sun, and Yingfei Xiong. Reinforcement learning
553 with negative tests as completeness signal for formal specification synthesis. *arXiv preprint*
554 *arXiv:2604.05820*, 2026.

555 Adharsh Kamath, Aditya Senthilnathan, Saikat Chakraborty, Pantazis Deligiannis, Shuvendu K.
556 Lahiri, Akash Lal, Aseem Rastogi, Subhajit Roy, and Rahul Sharma. Finding inductive loop
557 invariants using large language models. *arXiv preprint arXiv:2311.07948*, 2023.

558 Michael Karr. Affine relationships among variables of a program. *Acta Informatica*, 6(2):133–151,
559 1976.

560 Florent Kirchner, Nikolai Kosmatov, Virgile Prevosto, Julien Signoles, and Boris Yakobowski.
561 Frama-c: A software analysis perspective. *Formal Aspects of Computing*, 27(3):573–609, 2015.

562 Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C. H. Hoi. CodeRL:
563 Mastering code generation through pretrained models and deep reinforcement learning. In *Proc.*
564 *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

565 Chang Liu, Xiwei Wu, Yuan Feng, Qinxiong Cao, and Junchi Yan. Towards general loop invariant
566 generation: A benchmark of programs with memory manipulation. In *Proc. Advances in Neural*
567 *Information Processing Systems (NeurIPS)*, 2024a.

568 Ruibang Liu, Minyu Chen, Ling-I Wu, Jingyu Ke, and Guoqiang Li. Enhancing automated loop invari-
569 ant generation for complex programs with large language models. *arXiv preprint arXiv:2412.10483*,
570 2024b.

571 Lezhi Ma, Shangqing Liu, Yi Li, Xiaofei Xie, and Lei Bu. SpecGen: Automated generation of formal
572 program specifications via large language models. In *Proc. Int. Conf. on Software Engineering*
573 *(ICSE)*, 2025.

574 Kenneth L. McMillan. Interpolation and SAT-based model checking. In *Proc. Int. Conf. on Computer*
575 *Aided Verification (CAV)*, pp. 1–13. Springer, 2003.

576 Ido Pinto, Yizhak Yisrael Elboher, Haoze Wu, Nina Narodytska, and Guy Katz. Not all invariants
577 are equal: Curating training data to accelerate program verification with SLMs. *arXiv preprint*
578 *arXiv:2603.15510*, 2026.

579 Gabriel Ryan, Justin Wong, Jianan Yao, Ronghui Gu, and Suman Jana. CLN2INV: Learning loop
580 invariants with continuous logic networks. In *Proc. Int. Conf. on Learning Representations (ICLR)*,
581 2020.

582 Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang,
583 Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of
584 mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

585 Xujie Si, Hanjun Dai, Mukund Raghothaman, Mayur Naik, and Le Song. Learning loop invariants
586 for program verification. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*,
587 pp. 7751–7762, 2018.

-
- Xujie Si, Aaditya Naik, Hanjun Dai, Mayur Naik, and Le Song. Code2Inv: A deep learning framework for program verification. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 151–164. Springer, 2020.
- Christian Walder and Deep Karkhanis. Pass@k policy optimization: Solving harder reinforcement learning problems. In *Advances in Neural Information Processing Systems*, volume 38, 2025.
- Anjiang Wei, Tarun Suresh, Tianran Sun, Haoze Wu, Ke Wang, and Alex Aiken. InvBench: Can LLMs accelerate program verification with invariant synthesis? *arXiv preprint arXiv:2509.21629*, 2025.
- Cheng Wen, Jialun Cao, Jie Su, Zhiwu Xu, Shengchao Qin, Mengda He, Haokun Li, Shing-Chi Cheung, and Cong Tian. Enchanting program specification synthesis by large language models using static analysis and program verification. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 302–328. Springer, 2024.
- Guangyuan Wu, Weining Cao, Yuan Yao, Hengfeng Wei, Taolue Chen, and Xiaoxing Ma. LLM meets bounded model checking: Neuro-symbolic loop invariant inference. In *Proc. IEEE/ACM Int. Conf. on Automated Software Engineering (ASE)*, 2024a.
- Haoze Wu, Clark Barrett, and Nina Narodytska. Lemur: Integrating large language models in automated program verification. In *Proc. Int. Conf. on Learning Representations (ICLR)*, 2024b.
- Chuanhao Yan, Fengdi Che, Xuhan Huang, Xu Xu, Xin Li, Yizhi Li, Xingwei Qu, Jingzhe Shi, Chenghua Lin, Yaodong Yang, Binhang Yuan, Hang Zhao, Yu Qiao, Bowen Zhou, and Jie Fu. Re:Form—reducing human priors in scalable formal software verification with RL in LLMs: A preliminary study on Dafny. *arXiv preprint arXiv:2507.16331*, 2025.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Fanpeng Yang, Xing Li, Shuling Wang, Jie An, Zeyu Sun, Shenghua Feng, Wenhan Wang, Weiyi Wang, Naijun Zhan, and Fanjiang Xu. How powerful are LLMs in generating formal program specifications? In *Proc. Int. Conf. on Machine Learning (ICML)*, 2026a.
- Fanpeng Yang, Xu Ma, Shuling Wang, Xiong Xu, Qinxiang Cao, Naijun Zhan, Xiaofeng Li, and Bin Gu. Integrating symbolic execution with LLMs for automated generation of program specifications. *arXiv preprint arXiv:2506.09550*, 2026b.
- Jianan Yao, Gabriel Ryan, Justin Wong, Suman Jana, and Ronghui Gu. Learning nonlinear loop invariants with gated continuous logic networks. In *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pp. 106–120, 2020.
- Shiwen Yu, Ting Wang, and Ji Wang. Loop invariant inference through SMT solving enhanced reinforcement learning. In *Proc. ACM SIGSOFT Int. Symp. on Software Testing and Analysis (ISSTA)*, pp. 175–187, 2023. doi: 10.1145/3597926.3598047.
- Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? In *Advances in Neural Information Processing Systems*, volume 38, 2025.

A COMPLETE FORMALIZATION OF CRAFT

This section expands the method in Section 4 into a complete input-to-output specification. Given a task with hidden goals, CRAFT masks those goals and applies the same rollout–decompose–pool–filter–compose operator at inference time and during target construction. We first define the program semantics and verifier, then the exact and resource-bounded filters, and finally the inference, SFT, and RL procedures. Theoretical properties are stated and proved separately in Appendix B.

A.1 TASK SEMANTICS, LOOP INVARIANTS, AND VERIFICATION

Let Σ be the program-state space. A loop is $\mathcal{L} = (\text{Init}, B, S)$, where Init characterizes the first loop-head states, B is the guard, and $S \subseteq \Sigma \times \Sigma$ is one normally completing loop step. Thus $S(\sigma, \sigma')$ includes execution of the body and any guard or update side effects required to reach the next loop head.

Definition 1 (Inductive invariant). *A predicate I is a valid loop invariant when*

$$\begin{aligned} \forall \sigma \in \Sigma : \text{Init}(\sigma) \Rightarrow I(\sigma), \\ \forall \sigma, \sigma' \in \Sigma : I(\sigma) \wedge B(\sigma) \wedge S(\sigma, \sigma') \Rightarrow I(\sigma'). \end{aligned} \quad (7)$$

We denote these two obligations by $\text{Ind}_{\mathcal{L}}(I)$. Here and throughout the paper, valid invariant means inductive in this sense. This implies validity on every reachable loop-head state, although the converse need not hold for the chosen assertion language and prover.

A response is decomposed into a finite candidate family C of clauses. The clauses compose by conjunction,

$$I(C) = \bigwedge_{c \in C} c, \quad I(\emptyset) = \text{true}. \quad (8)$$

The set C is *jointly inductive* when $\text{Ind}_{\mathcal{L}}(I(C))$ holds. Notice that preservation is checked under the full conjunction: a clause may rely on companion clauses. The logical definition is insensitive to clause order. The bounded verifier, however, receives a deterministic sequence $D = \langle d_1, \dots, d_m \rangle$, because generated proof dependencies can follow annotation order. Set notation below denotes the members of this sequence; deletion preserves their relative order.

A complete task is $\mathcal{T} = (\mathcal{L}, \mathcal{G})$, where $\mathcal{G}$ contains the assertions and postconditions at their original program locations. For a body target, let $S_{\ell}^B \subseteq \Sigma \times \Sigma$ be the path-sensitive prefix relation from body entry to ℓ . For a target after the loop, let S_{ℓ}^E be the suffix relation from loop exit to ℓ , with the identity relation used when the target is immediately after the loop. Each relation is the union of its feasible paths and therefore includes their branch conditions. Define

$$\text{VC}_{\mathcal{L}}(I, \ell, g) = \begin{cases} \forall \sigma, \sigma' : I(\sigma) \wedge B(\sigma) \wedge S_{\ell}^B(\sigma, \sigma') \Rightarrow g(\sigma'), & \ell \text{ is inside the body,} \\ \forall \sigma, \sigma' : I(\sigma) \wedge \neg B(\sigma) \wedge S_{\ell}^E(\sigma, \sigma') \Rightarrow g(\sigma'), & \ell \text{ is after the loop.} \end{cases} \quad (9)$$

Postconditions are represented by the second case, with the suffix relation including assignments and return-value substitution. The ideal logical verdict is

$$\text{TaskValid}_{\models}(\mathcal{T}, C) = \text{Ind}_{\mathcal{L}}(I(C)) \wedge \bigwedge_{(\ell, g) \in \mathcal{G}} \text{VC}_{\mathcal{L}}(I(C), \ell, g). \quad (10)$$

Generation and training see only $\mathcal{L}$; $\mathcal{G}$ is restored only for Equation (10).

For a proof goal g generated by the configured loop verifier, define its portfolio status by

$$\text{GoalStatus}_{\delta}(g) \in \{\text{valid}, \text{timeout}, \text{unknown}\}, \quad (11)$$

where valid means that at least one configured prover returned *Valid*; timeout means that no prover proved the goal and at least one exhausted δ ; and unknown contains every other non-valid, unsupported, missing, or malformed result.

Let $\mathcal{S}_{\text{ver}} = \{\text{valid}, \text{timeout}, \text{unknown}\}$. One invocation of the resource-bounded loop verifier on the current ordered family $D = \langle d_1, \dots, d_m \rangle$ returns the clause-to-status map

$$\text{LoopVerify}_{\mathcal{L}, \delta}(D) = \{d_k \mapsto v_{D,k} : 1 \leq k \leq m\} \in \mathcal{S}_{\text{ver}}^D, \quad (12)$$

where $v_{D,k} = \text{valid}$ means that the verifier proves all establishment and preservation obligations generated for d_k in the current family D . If the obligations are not all proved, $v_{D,k} = \text{timeout}$ records an exhausted prover budget and $v_{D,k} = \text{unknown}$ records every other outcome. The interface is sound when

$$(\forall c \in D : \text{LoopVerify}_{\mathcal{L},\delta}(D)[c] = \text{valid}) \implies \text{Ind}_{\mathcal{L}}(I(D)). \quad (13)$$

The filter treats timeout and unknown identically, but the interface keeps them distinct. Appendix D.1 describes how the implementation generates and aggregates the clause goals.

After the invariant set is fixed, the separate binary operator $\text{TaskVerify}_{\delta}(\mathcal{T}, C)$ runs the verifier on the original task annotated with C . Let $\text{TargetGoals}(\mathcal{T}, C)$ be its generated source-level assertion and postcondition goals. Then

$$\begin{aligned} \text{TaskVerify}_{\delta}(\mathcal{T}, C) = & \mathbf{1}[\text{SyntaxOK}(\mathcal{T}, C) \wedge \\ & \forall c \in C : \text{LoopVerify}_{\mathcal{L},\delta}(C)[c] = \text{valid} \wedge \\ & \forall g \in \text{TargetGoals}(\mathcal{T}, C) : \text{GoalStatus}_{\delta}(g) = \text{valid}]. \end{aligned} \quad (14)$$

A.2 IDEAL AND RESOURCE-BOUNDED HOUDINI FILTERING

We define an exact semantic fixed-point operator and its resource-bounded verifier counterpart.

For $c \in C$, define its establishment and context-dependent preservation obligations by

$$\begin{aligned} \text{Est}_{\mathcal{L}}(c) & \iff \forall \sigma : \text{Init}(\sigma) \Rightarrow c(\sigma), \\ \text{Pres}_{\mathcal{L},C}(c) & \iff \forall \sigma, \sigma' : I(C)(\sigma) \wedge B(\sigma) \wedge S(\sigma, \sigma') \Rightarrow c(\sigma'). \end{aligned} \quad (15)$$

Ideal Houdini starts with $C_0 = C$ and repeats

$$C_{j+1} = F_{\mathcal{L}}(C_j) = \{c \in C_j : \text{Est}_{\mathcal{L}}(c) \wedge \text{Pres}_{\mathcal{L},C_j}(c)\} \quad (16)$$

until a fixed point $H_{\mathcal{L}}(C)$ is reached.

The implemented deletion round uses the clause-level loop verifier and preserves the relative order of retained clauses:

$$\widehat{F}_{\mathcal{L},\delta}(D) = \langle c \in D \mid \text{LoopVerify}_{\mathcal{L},\delta}(D)[c] = \text{valid} \rangle. \quad (17)$$

Thus timeout and unknown both cause conservative deletion. Every status in one round is computed under the same pre-deletion context D . A clause removed at the end of that round may therefore have helped another clause receive valid. The survivors must be verified again without the removed hypotheses; this is why a single deletion pass is insufficient. For an ordered input sequence D , let $D_0 = D$ and $D_{j+1} = \widehat{F}_{\mathcal{L},\delta}(D_j)$. Iteration stops at the first proved fixed point or at the symbolic round limit $h_{\max}$. Let

$$j^* = \min\{j < h_{\max} : \widehat{F}_{\mathcal{L},\delta}(D_j) = D_j\}, \quad (18)$$

when this set is nonempty. The implemented filter is

$$\text{Filter}_{\mathcal{L}}^{\delta, h_{\max}}(D) = \begin{cases} D_{j^*}, & j^* \text{ exists,} \\ \langle \rangle, & \text{the round limit is reached first.} \end{cases} \quad (19)$$

At a returned fixed point, every surviving clause has just been proved in the context of that same sequence. Reaching the limit returns the empty sequence. Under a sound loop verifier, the result is sound and may be smaller than the ideal fixed point. In the remainder, $\text{Filter}_{\mathcal{L}}$ abbreviates Equation (19), and TaskVerify abbreviates $\text{TaskVerify}_{\delta}$.

A.3 THE SHARED COMPOSITION OPERATOR

Let $\text{parts}(y)$ extract the ordered sequence of invariant clauses in response y , let $\text{First}_m(L)$ be the prefix of sequence L containing at most m clauses, and let $\parallel$ denote sequence concatenation. Define the clauses admitted from one response by

$$\text{Admit}(y) = \text{Unique}(\text{First}_{m_{\max}}(\text{parts}(y))). \quad (20)$$

Unique removes duplicates using the finite-rule canonical normalization $\text{key}(c)$. Equal keys identify the clauses merged by those rules. Provenance is retained by recording which responses supplied

each key. It keeps the first occurrence; concatenations are traversed in response order and then clause order, which fixes the sequence supplied to LoopVerify. For a clause sequence D , let $\text{keys}(D) = \langle \text{key}(c) : c \in D \rangle$.

For any response sequence $y_{1:n}$, define

$$A_i = \text{Admit}(y_i), \quad P(y_{1:n}) = \text{Unique}(A_1 \parallel \dots \parallel A_n), \quad (21)$$

$$C(y_{1:n}) = \text{Filter}_{\mathcal{L}}(P(y_{1:n})), \quad I(y_{1:n}) = \bigwedge_{c \in C(y_{1:n})} c. \quad (22)$$

This is the shared rollout–decompose–pool–filter–compose operator. The filter checks preservation under the complete current conjunction, so clauses from different responses may support one another; standalone preservation is not required.

A.4 COMPLETE INFERENCE ALGORITHM

Algorithm 2 CRAFT inference

Input: task $\mathcal{T} = (\mathcal{L}, \mathcal{G})$, policy π , number of responses n
Output: composed invariant I , final verdict v

- 1: $x \leftarrow \text{Mask}(\mathcal{T}) = \mathcal{L} \triangleright \text{hide goals}$
- 2: sample $y_{1:n} \sim \pi(\cdot \mid x) \triangleright \text{rollout}$
- 3: $A_i \leftarrow \text{Admit}(y_i)$ for every $i \triangleright \text{decompose and cap}$
- 4: $P \leftarrow \text{Unique}(A_1 \parallel \dots \parallel A_n) \triangleright \text{pool}$
- 5: $C \leftarrow \text{Filter}_{\mathcal{L}}^{\delta, h_{\max}}(P) \triangleright \text{filter}$
- 6: $I \leftarrow \bigwedge_{c \in C} c \triangleright \text{compose}$
- 7: $v \leftarrow \text{TaskVerify}_{\delta}(\mathcal{T}, C) \triangleright \text{restore goals}$
- 8: **return** (I, v)

Compose@ n estimates $\Pr[\text{TaskVerify}(\mathcal{T}, C(y_{1:n}))]$.

A.5 COMPLETE SFT TARGET CONSTRUCTION

SFT distills the same multi-response map into one response. It accumulates raw candidates so that a clause discarded in one round can later acquire a supporting companion. The policy π_0 is the small open-weight model being trained and is the only learned generator in target construction. Executions of $\mathcal{L}$ produce the trace signal, and Filter $_{\mathcal{L}}$ supplies the verifier decision; no external model supplies candidates, labels, or acceptance judgments.

Algorithm 3 Multi-response SFT synthesis

Input: masked loop $\mathcal{L}$, policy π_0 , negatives $\mathcal{N}$, group size g_{sft} , round limit t_{sft}
Output: one serialized target, or reject

- 1: **if** $\mathcal{N} = \emptyset$ **then return** reject
- 2: $P \leftarrow \langle \rangle$
- 3: **for** $t = 1, \dots, t_{\text{sft}}$ **do**
- 4: sample a response group $y_{1:g_{\text{sft}}} \sim \pi_0(\cdot \mid \mathcal{L})$
- 5: $P \leftarrow \text{Unique}(P \parallel \text{Admit}(y_1) \parallel \dots \parallel \text{Admit}(y_{g_{\text{sft}}}))$
- 6: $C \leftarrow \text{Filter}_{\mathcal{L}}(P)$
- 7: **if** $\text{cov}_{\mathcal{N}}(C) \geq \tau$ **then return** $\text{Serialize}(C)$
- 8: **return** reject

The SFT input and target contain no element of $\mathcal{G}$. The parameter values appear in Appendix D.

A.6 COMPLETE RL GROUP-SCORING ALGORITHM

The sampled policy is the same small open-weight model updated by RL. Given its rollouts, every reward component below is computed from program executions, verifier outcomes, and deterministic clause provenance; no external model enters the scoring function.

Executions of the masked loop yield reachable loop-head states $\mathcal{E}$ and target-independent negative traces $\mathcal{N}$. The lightweight evaluator is three-valued:

$$\text{Eval}(c, s) \in \{\text{true}, \text{false}, \text{unknown}\}. \quad (23)$$

An unknown result neither rejects a negative trace nor removes a candidate in the reachable-state front end; syntax and induction are left to the verifier. A trace counts as rejected only when at least one retained clause evaluates definitively to false at one of its states:

$$\text{rej}_{\mathcal{N}}(C) = \{\nu \in \mathcal{N} : \exists s \in \nu, \exists c \in C, \text{Eval}(c, s) = \text{false}\}, \quad \text{cov}_{\mathcal{N}}(C) = \frac{|\text{rej}_{\mathcal{N}}(C)|}{|\mathcal{N}|}. \quad (24)$$

Coverage is trajectory level and is defined only for $|\mathcal{N}| > 0$. Instances without retained negatives are rejected during SFT construction. For RL, the explicit fallback below uses a binary survivor signal and never evaluates this ratio.

For an RL group $y_{1:g_{\text{rl}}}$, first form the same globally de-duplicated pool used at inference, and then invoke the verifier filter once:

$$\begin{aligned} L_i &= \text{parts}(y_i), & o_i &= \max(0, |L_i| - m_{\text{max}}), \\ A_i &= \text{Admit}(y_i), & P^* &= \text{Unique}(A_1 \parallel \dots \parallel A_{g_{\text{rl}}}), \end{aligned} \quad (25)$$

$$C^* = \text{Filter}_{\mathcal{L}}(P^*), \quad V_i = \langle c \in A_i \mid \text{key}(c) \in \text{keys}(C^*) \rangle, \quad R_i = \text{rej}_{\mathcal{N}}(V_i). \quad (26)$$

Here V_i assigns each pooled survivor back to every response that proposed it. A reachable-state front end evaluates the globally pooled candidates on $\mathcal{E}$ before the verifier processes the same pool. If $f(\nu) = |\{j : \nu \in R_j\}|$, the reward is

$$r_i = w_c \frac{|R_i|}{|\mathcal{N}|} + w_g \left(\frac{1}{|\mathcal{N}|} \sum_{\nu \in R_i} \frac{1}{f(\nu)} \right) - w_o o_i. \quad (27)$$

Here w_c , w_g , and w_o are the coverage, group-credit, and overflow weights. The first term rewards each response by the negative coverage of its pooled survivors. The second is the Shapley value of the fixed post-filter coverage game $u(A) = |\bigcup_{i \in A} R_i|/|\mathcal{N}|$: repeated coverage is shared among its proposers, while unique coverage receives full credit. A purely supporting clause may enable another clause to survive but receives no direct coverage credit. The final term charges each clause beyond the per-response cap.

Algorithm 4 CRAFT reward for one RL group

Input: masked loop $\mathcal{L}$, responses $y_{1:g_{\text{rl}}}$, reachable states $\mathcal{E}$, negative traces $\mathcal{N}$
Output: response rewards $r_{1:g_{\text{rl}}}$

- 1: $L_i \leftarrow \text{parts}(y_i)$; $o_i \leftarrow \max(0, |L_i| - m_{\text{max}})$ for every i
- 2: $A_i \leftarrow \text{Admit}(y_i)$ for every i
- 3: $P^* \leftarrow \text{Unique}(A_1 \parallel \dots \parallel A_{g_{\text{rl}}}) \triangleright \text{pool globally}$
- 4: $C^* \leftarrow \text{Filter}_{\mathcal{L}}(P^*) \triangleright \text{filter the pool once}$
- 5: $V_i \leftarrow \langle c \in A_i \mid \text{key}(c) \in \text{keys}(C^*) \rangle$ for every $i \triangleright \text{project by provenance}$
- 6: **if** $\mathcal{N} = \emptyset$ **then**
- 7: $r_i \leftarrow \mathbf{1}[A_i \neq \langle \rangle \wedge \text{keys}(V_i) = \text{keys}(A_i)]$ for every i
- 8: **return** $r_{1:g_{\text{rl}}}$
- 9: $R_i \leftarrow \text{rej}_{\mathcal{N}}(V_i)$ for every i
- 10: $f(\nu) \leftarrow |\{j : \nu \in R_j\}|$ for every $\nu \in \mathcal{N}$
- 11: $r_i \leftarrow w_c |R_i|/|\mathcal{N}| + w_g |\mathcal{N}|^{-1} \sum_{\nu \in R_i} f(\nu)^{-1} - w_o o_i$ for every i
- 12: **return** $r_{1:g_{\text{rl}}}$

A clause c *semantically supports* d in D when $c \neq d$, $\text{Pres}_{\mathcal{L},D}(d)$ holds, but $\text{Pres}_{\mathcal{L},D \setminus \{c\}}(d)$ does not. The implemented verifier exposes the broader, resource-dependent relation

$$\text{VSupport}_{\mathcal{L},\delta}(c, d \mid D) \iff c \neq d \wedge \text{LoopVerify}_{\mathcal{L},\delta}(D)[d] = \text{valid} \wedge \text{LoopVerify}_{\mathcal{L},\delta}(D \setminus \{c\})[d] \neq \text{valid}. \quad (28)$$

This relation includes semantic support, generated-goal dependencies, and resource effects. Algorithm 4 can retain such a clause through joint filtering, but assigns it direct reward only if its projected response rejects a negative trace.

A.7 GROUP-RELATIVE POLICY UPDATE

The rollout policy $\pi_{\theta_{\text{old}}}$ samples a group, which is scored by Algorithm 4. Define

$$\hat{A}_i = \frac{r_i - \text{mean}_j r_j}{\text{std}_j r_j + \delta_{\text{adv}}}. \quad (29)$$

For token $a_{i,u}$ and its prefix $h_{i,u} = (\mathcal{L}, a_{i,<u})$, let

$$\rho_{i,u}(\theta) = \frac{\pi_{\theta}(a_{i,u} \mid h_{i,u})}{\pi_{\theta_{\text{old}}}(a_{i,u} \mid h_{i,u})}, \quad (30)$$

$$q_{i,u}(\theta) = \frac{\pi_{\text{ref}}(a_{i,u} \mid h_{i,u})}{\pi_{\theta}(a_{i,u} \mid h_{i,u})}, \quad k_{i,u}(\theta) = q_{i,u} - \log q_{i,u} - 1. \quad (31)$$

The nonnegative quantity $k_{i,u}$ is the sampled conditional-token KL surrogate used by the implementation. Its expectation equals $D_{\text{KL}}(\pi_{\theta}(\cdot \mid h_{i,u}) \parallel \pi_{\text{ref}}(\cdot \mid h_{i,u}))$ when the token is sampled from $\pi_{\theta}(\cdot \mid h_{i,u})$. During an update, however, the saved token comes from $\pi_{\theta_{\text{old}}}$, so $k_{i,u}$ is used as a sampled penalty rather than claimed to be an unbiased off-policy estimate. The clipped group objective is

$$\mathcal{J}(\theta) = \mathbb{E} \left[\frac{1}{g_{\text{rl}}} \sum_i \frac{1}{T_i} \sum_{u=1}^{T_i} \left(\min \left\{ \rho_{i,u} \hat{A}_i, \text{clip}(\rho_{i,u}, 1 - \varepsilon, 1 + \varepsilon) \hat{A}_i \right\} - \beta k_{i,u} \right) \right]. \quad (32)$$

After optimizing this objective, the updated policy samples the next group.

A.8 TRAINING–INFERENCE ALIGNMENT

Inference, SFT target construction, and RL group scoring all call the same pooled map in Equation (22); they differ only in how the returned clauses are consumed. Inference checks the restored goals, SFT serializes an accepted fixed point, and RL projects the fixed point back to the responses that proposed its surviving clauses. Negative coverage supplies the target-independent training surrogate; restored-goal verification supplies the final verdict.

A.9 EXTENSION TO VERIFIABLY COMPOSABLE GENERATION

Definition 2 (Verifiably composable generation). *A task provides visible input x , hidden goals $\mathcal{G}$, an atomic decomposition parts_x , a duplicate-removal operator Unique_x , a subset filter Filter_x , a composition operator Compose_x , and a validity predicate Valid_x . For responses $y_{1:n}$, define*

$$P_x(y_{1:n}) = \text{Unique}_x \left(\bigcup_i \text{parts}_x(y_i) \right), \quad (33)$$

$$O_x(y_{1:n}) = \text{Compose}_x(\text{Filter}_x(P_x(y_{1:n}))). \quad (34)$$

The task is verifiably composable when the filter is conservative and sound: it returns atoms from its input and $\text{Valid}_x(O_x(y_{1:n}))$ holds whenever the filter succeeds. It is target hidden when every operator above depends only on x ; the goals $\mathcal{G}$ are used only by a final task-specific check. Complementary composition occurs when $O_x(y_{1:n})$ passes that final check although no individual $O_x(y_i)$ does.

Loop-invariant generation instantiates the atoms with invariant clauses, composition with conjunction, validity with joint inductiveness, and the filter with bounded Houdini. The transfer property for any other instantiation satisfying Definition 2 is proved in Appendix B.

Definition 2 identifies the conditions needed to reuse CRAFT: responses must decompose into reusable atoms, atoms must admit pooling, a sound domain-specific filter must validate the pooled result, and provenance must map accepted atoms back to responses. Under these conditions, the same inference and training construction applies after replacing the invariant clauses, conjunction, and Houdini checks by the domain’s atoms, composition operator, and verifier.

For *specification generation*, atoms can be precondition, postcondition, frame, or invariant clauses; composition is conjunction; and the filter checks syntax, consistency, and the corresponding proof obligations. Separate responses can contribute complementary clauses to one verified contract. For *test-case generation*, atoms are executable tests; composition is set union; the filter removes invalid or duplicate tests; and structural or behavioral coverage supplies a target-independent signal. Separate responses can cover disjoint behaviors even when no one response forms an adequate suite.

These domains share the formal interface and algorithms. Each domain supplies its verifier and measurable training signal.

B THEORETICAL PROPERTIES AND PROOFS

This section states and proves the properties used by the complete algorithms in Appendix A. Write $\mathcal{L} = (\text{Init}, B, S)$.

B.1 CONJUNCTION OF VALID INVARIANTS

Proposition 3 (Closure under conjunction). *If $I_1, \dots, I_m$ are valid invariants of the same loop, then $\bigwedge_j I_j$ is valid and is at least as strong as every I_j .*

Proof of Proposition 3. For each j , validity gives $\text{Init} \Rightarrow I_j$; hence $\text{Init} \Rightarrow \bigwedge_j I_j = I$. Suppose $I \wedge B$ holds before one execution of S . Then $I_j \wedge B$ holds for every j , so preservation of I_j establishes every I_j afterward. Therefore I is preserved and is a valid invariant.

Because a conjunction implies each conjunct, I is at least as strong as every I_j . $\square$

B.2 MUTUAL SUPPORT AMONG ARBITRARILY MANY CLAUSES

Proposition 4 (Mutual support). *For a finite clause set C , $I(C)$ is valid if*

$$\text{Init} \Rightarrow I(C), \quad \forall c \in C : \text{Pres}_{\mathcal{L}, C}(c). \quad (35)$$

No clause must be preserved in isolation.

Proof of Proposition 4. The first premise is exactly establishment of $I(C)$. For preservation, assume $I(C) \wedge B$ before executing S . The second premise establishes each c_j afterward. Since every conjunct then holds in the same successor state, their conjunction $I(C)$ also holds. Thus $\{I(C) \wedge B\} S \{I(C)\}$, and $I(C)$ is valid.

No standalone preservation premise appears in this argument. A clause may use all other conjuncts in the pre-state, so clauses that fail preservation separately may still be preserved jointly. Establishment is different: $\text{Init} \Rightarrow I(C)$ implies $\text{Init} \Rightarrow c_j$ for every j . Hence a candidate false at some loop-entry state cannot be rescued by conjunction. $\square$

B.3 GREATEST JOINTLY INDUCTIVE CANDIDATE SUBSET

Let $C_0 = C$, and let $C_{j+1} \subseteq C_j$ be the set remaining after one ideal Houdini deletion round in Equation (16). Because C is finite, this descending sequence reaches the fixed point $H_{\mathcal{L}}(C)$.

Theorem 5 (Candidate-relative optimality). *With exact logical decisions, $H_{\mathcal{L}}(C)$ is jointly inductive and contains every jointly inductive subset of C . Its conjunction is therefore the strongest invariant obtainable by selecting clauses from this pool.*

Proof of Theorem 5. At the fixed point, every $c \in H_{\mathcal{L}}(C)$ satisfies establishment and preservation under the full conjunction $I(H_{\mathcal{L}}(C))$. Applying Proposition 4 shows that this conjunction is valid.

It remains to prove greatestness. Let $C' \subseteq C$ be any jointly inductive subset. We show by induction over deletion rounds that $C' \subseteq C_j$. The base case $C' \subseteq C_0 = C$ is immediate. Assume $C' \subseteq C_j$, and take any $c \in C'$. Joint establishment of C' gives $\text{Init} \Rightarrow c$, so c cannot be deleted for establishment. Joint preservation gives

$$\{I(C') \wedge B\} S \{c\}.$$

Since $C' \subseteq C_j$, the antecedent $I(C_j) \wedge B$ implies $I(C') \wedge B$. Strengthening a valid Hoare precondition preserves validity, so

$$\{I(C_j) \wedge B\} S \{c\}$$

also holds. Therefore no member of C' is deleted in round j , proving $C' \subseteq C_{j+1}$. At termination, $C' \subseteq H_{\mathcal{L}}(C)$. Because conjunction reverses set inclusion, $I(H_{\mathcal{L}}(C))$ implies $I(C')$. $\square$

B.4 SOUNDNESS OF RESOURCE-BOUNDED HOUDINI

Proposition 6 (Bounded-filter soundness). *Assume $\text{LoopVerify}_{\mathcal{L},\delta}$ is sound. If $\text{Filter}_{\mathcal{L}}^{\delta, h_{\max}}(C)$ returns a proved fixed point D , then $D \subseteq C$ and $I(D)$ is inductive. Timeout, unknown, and exhaustion of the round budget cannot cause an unproved clause set to be returned.*

Proof. Every deletion round in Equation (17) selects a subset of its input, so $D \subseteq C$. At a returned fixed point, every clause maps to valid in that same family. Verifier soundness in Equation (13) therefore gives $\text{Ind}_{\mathcal{L}}(I(D))$. A timeout or unknown removes the affected clause, and reaching $h_{\max}$ returns the empty result rather than an intermediate set by Equation (19). $\square$

B.5 CLOSED FORM OF THE COVERAGE-GAME SHAPLEY VALUE

The allocation of Equation (5) is the Shapley value of the coverage game $u(\mathcal{S}) = |\bigcup_{i \in \mathcal{S}} R_i|/|\mathcal{N}|$, for coalitions $\mathcal{S} \subseteq \mathcal{R}$ of the rollout index set.

Proposition 7 (Closed form of the coverage game). *For the coverage game $u(\mathcal{S}) = |\bigcup_{i \in \mathcal{S}} R_i|/|\mathcal{N}|$, the Shapley value is $\phi_i = \frac{1}{|\mathcal{N}|} \sum_{\nu \in R_i} \frac{1}{f(\nu)}$, and $\sum_i \phi_i = u(\mathcal{R})$.*

Proof. Fix a trace ν rejected by $f(\nu)$ rollouts. In a uniformly random ordering of the group, ν contributes to the marginal gain of exactly one player—the first of those $f(\nu)$ rollouts to appear—and by symmetry each of them is first with probability $1/f(\nu)$. Player i therefore receives expected credit $1/f(\nu)$ for each $\nu \in R_i$ and zero otherwise. Summing over $\nu \in \mathcal{N}$ and normalizing by $|\mathcal{N}|$ gives the closed form; efficiency follows because each covered trace distributes exactly one unit of credit among its $f(\nu)$ rejectors. $\square$

B.6 SUPPORTING CLAUSES AND CREDIT

Proposition 8 (Coverage credit omits pure support). *Suppose response i contributes a retained clause c that supports another retained clause in the sense of Appendix A, but $\text{rej}_{\mathcal{N}}(V_i) = \emptyset$. Then both coverage terms assigned to response i are zero, irrespective of whether removing c would prevent the other clause from surviving.*

Proof. The premise gives $R_i = \emptyset$. Therefore $|R_i| = 0$ and the sum $\sum_{\nu \in R_i} 1/f(\nu)$ is empty, so Equation (27) assigns zero to both terms. Filter dependence does not appear in the fixed coverage game after V_i has been formed. $\square$

B.7 TRANSFER TO VERIFIABLY COMPOSABLE PROBLEMS

Proposition 9 (Composable transfer). *For any instantiation satisfying Definition 2, the pooled output $O_x(y_{1:n})$ contains only atoms drawn from the responses and satisfies Valid_x whenever filtering succeeds. Moreover, its construction is independent of the hidden goals $\mathcal{G}$.*

Proof. Decomposition and union draw atoms only from $y_{1:n}$; Unique_x removes candidates and the conservative filter returns a subset, establishing provenance to the responses. Filter soundness gives $\text{Valid}_x(O_x(y_{1:n}))$. Finally, every operator in Equation (34) is a function of x and the responses, not $\mathcal{G}$, so hidden goals enter only the final check. $\square$

C SCOPE AND MOTIVATING EXAMPLE

We focus on numerical loops because their arithmetic equalities and inequalities, including polynomial relations, admit scalable automatic checking. The study considers one braced while loop over scalar C integers, including nonlinear updates. Even this setting can hide nonlinear relations across coupled recurrences. For example:

```
1 void f(int k, int z) {
2   int x = 1, y = 1, c = 1;
3   while (c < k) { c++; x = x*z + 1; y = y*z; }
4   /*@ assert 1+x*z-x-z*y == 0; */
5 }
```

Case study: target-hidden invariant discovery. If the assertion is visible, it can be copied as a loop-invariant candidate. It holds initially because $x = y = 1$. Writing $I(x, y) = 1 + xz - x - zy$, one loop iteration gives

$$I(xz + 1, yz) = 1 + (xz + 1)z - (xz + 1) - z(yz) = zI(x, y),$$

so $I = 0$ is inductive and discharges the exit assertion. Under target hiding, the model sees the initialization and recurrences but not the assertion, and must recover the relation. Archived rollout pools recover three algebraically equivalent forms:

Model	First verified prefix	Rediscovered invariant clause
GPT-5	compose@1	$x - 1 = z(x - y)$
DeepSeek V4-Flash	compose@4	$x(z - 1) = zy - 1$
Claude Sonnet-4.6	compose@8	$x(z - 1) = yz - 1$

All three clauses rearrange to the hidden assertion. None of the ten raw responses in these pools passes as a whole on this program; pooled filtering removes incompatible siblings and retains the useful relation.

D IMPLEMENTATION DETAILS

Table 3 lists the implemented defaults. SFT synthesis, RL, and inference share Algorithm 1. RL filters the union once, then uses clause provenance to assign survivors back to their proposing rollouts before computing credit (Appendix M).

The fixed near-input tier order is $\langle 0, 1, 2, -1, 3, 5, 8, -2, 10, 17, 4, -3, 25, 6, 40, 7, -5, 13, 50, 9 \rangle$.

D.1 LOOP-VERIFIER REALIZATION

The implementation generates establishment and preservation goals for each clause in the ordered current family $D = \langle d_1, \dots, d_m \rangle$. Establishment of d_k may use earlier annotations d_j , $j < k$, as hypotheses. Preservation uses the full current family as loop-head hypotheses and may use earlier annotations in the successor state. These hypotheses are available during the current round regardless

Symbol	Value	Role
r_{exec}	12	requested input executions; doubled for oracle-driven bodies
$\mathcal{X}_{\text{tier}}$	20; $[-64, 64]$	fixed near-input schedule and clamping interval
q_{far}	3, plus 2 ordered multi-parameter probes	deterministic far probes with magnitude at most 512
Δ_{rel}	dense: $\pm\{1, 2\}$; terminal: $\pm\{1, 2, 3, 5, 8\}$	local relational perturbations
h_{dense}	3	successor window used to establish a relation base
b_{rel}	96	cap on relation bases stratified across positives
h_{exit}	24	continuation horizon after a witnessed exit
$b_{\text{rel}}^-, b_{\text{exit}}^-$	48, 12	relational and post-exit negative-trace budgets
u_{max}	2×10^6	execution-iteration safety cap
w_c, w_g, w_o	1.0, 0.3, 0.1	coverage, group-credit, and overflow weights
m_{max}	20 clauses	cap applied independently to each response
α	0.95	saturation fraction used to report k_α
$\mathcal{K}$	$\{1, 2, 4, 8, 16, 32\}$	reported Bare response budgets
n_{probe}	128; 100; 32	saved responses per task (8B; A3B/Llama; remaining base models)
ϑ	1.0	decoding temperature except for Loopy (Table 18)
p	1.0	nucleus-sampling probability
L_{api}	8,192 tokens	maximum API-model completion length
L_{local}	2,048 tokens	maximum local-model completion length
g_{sft}	16 responses	group sampled per SFT strengthening round
t_{sft}	5 rounds	maximum per loop; at most 80 responses
τ	0.5	SFT acceptance threshold on composed negative coverage
h_{max}	500	maximum Houdini verification rounds
$\mathcal{P}_{\text{verify}}$	Alt-Ergo and Z3	prover portfolio; Typed memory model
δ	5 seconds per goal	per-prover goal budget used by LoopVerify
$\delta_{\text{short}}, \delta_{\text{long}}$	300, 900 seconds	verification caps in the pre-/post-filtering comparison

Table 3: Parameter values for execution sampling (§4.2), SFT synthesis (§4.3), reward computation (§4.4), inference, and verification and evaluation.

of the status of their own goals. The verifier simplifies and splits the obligations by control-flow path. A clause receives valid only when every generated establishment and preservation subgoal is proved; timeout records an exhausted prover budget, and all other outcomes map to unknown. Houdini reruns the survivors after each deletion round.

D.2 NEGATIVE-TRACE SAMPLING

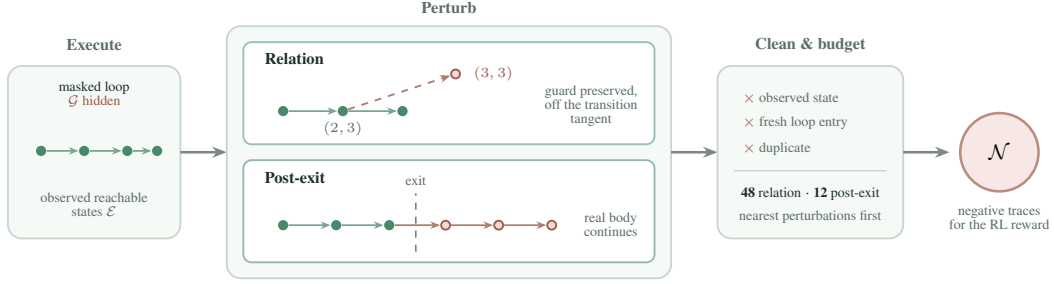


Figure 5: **Construction of target-hidden negative traces.** Concrete executions provide observed reachable states. Two transformations probe relational structure and behavior past a genuine exit. Conservative cleaning and per-family budgets produce $\mathcal{N}$, which is used only for RL reward computation and never exposes $\mathcal{G}$.

Algorithm 5 summarizes the sampler used for RL. Trace records real loop-head states and, after each genuine exit, continues the real loop body to obtain one post-exit trace. Perturb_Θ applies the configured single-variable and pairwise changes; the exact perturbations and budgets are those in Table 3.

The two families target different clause properties. A relation perturbation is admitted only when it stays inside the sampled envelope of its context, preserves the guard truth value, and does not lie on the locally witnessed transition tangent, emphasizing clauses that constrain relations among variables. A post-exit continuation executes the real body past a witnessed exit into states that no perturbation of an observed state can produce, emphasizing clauses that characterize the exit boundary. The sampler excludes witnesses rejected by single-variable range bounds or frame equalities. Section 5.6 ablates the structured families against uniform random perturbations.

Algorithm 5 Target-hidden negative-trace sampling

Input: masked loop $\mathcal{L} = (\text{Init}, B, S)$, input schedule U , configuration Θ
Output: reachable states $\mathcal{E}$, negative traces $\mathcal{N}$

- 1: $\mathcal{T}_{\text{exec}}, \mathcal{T}_{\text{post}}, \text{capped} \leftarrow \text{Trace}(\mathcal{L}, U, \Theta) \triangleright \text{capped: some run was capped}$
- 2: $\mathcal{E} \leftarrow \text{UniqueHeads}(\mathcal{T}_{\text{exec}})$; $M \leftarrow \text{SafeModifiedVars}(S)$
- 3: **if** $M = \emptyset$ or S has unsupported state **then return** $(\mathcal{E}, \emptyset)$
- 4: $\mathcal{E}_{\text{sample}} \leftarrow \text{StratifiedSample}(\mathcal{E})$
- 5: $C_{\text{rel}} \leftarrow \text{Perturb}_\Theta(\text{Dense}(\mathcal{E}_{\text{sample}}), M)$
- 6: **if** $\neg \text{capped}$ **then** $C_{\text{rel}} \leftarrow C_{\text{rel}} \cup \text{Perturb}_\Theta(\text{Terminals}(\mathcal{T}_{\text{exec}}), M)$
- 7: $C_{\text{rel}} \leftarrow \text{TangentAndEnvelopeFilter}(C_{\text{rel}}, \mathcal{T}_{\text{exec}})$
- 8: $(C_{\text{rel}}, \mathcal{T}_{\text{post}}) \leftarrow \text{Clean}_{\mathcal{E}}(C_{\text{rel}}, \mathcal{T}_{\text{post}})$
- 9: $\hat{C}_{\text{rel}}, \mathcal{N}_{\text{post}} \leftarrow \text{BudgetAndRoundRobin}(C_{\text{rel}}, \mathcal{T}_{\text{post}}, \Theta)$
- 10: $\mathcal{N} \leftarrow \{\langle s \rangle : s \in \hat{C}_{\text{rel}}\} \cup \mathcal{N}_{\text{post}}$
- 11: **return** $(\mathcal{E}, \mathcal{N})$

$\text{Clean}_{\mathcal{E}}$ removes observed reachable states, possible fresh loop entries, duplicates, and unsupported candidates; a cleaned post-exit continuation remains one trace, and relation candidates become singleton traces. Within each structural bucket the round-robin prefers the smallest counterfactual change, so surviving witnesses sit close to the sampled manifold.

D.3 CLAUSE NORMALIZATION AND OPTIMIZATION SETTINGS

Appendix A.7 defines the group-normalized advantage, token-level importance ratio, sampled conditional KL term, and clipped GRPO objective. Clause de-duplication, which forms the clause set scored above, normalizes comparison direction, equality sides, parentheses, immediate commutative

operands, harmless identities, and Boolean association, but avoids transformations that require additional arithmetic assumptions. Repeated clauses therefore collapse without affecting coverage, and a unique zero-coverage clause costs nothing because it may support another clause or the hidden target. Both coverage and Shapley credit lie in $[0, 1]$; the overflow term subtracts $w_o o_i$. Empty-negative groups use the binary fallback in Algorithm 4 without this penalty.

E GENERATION PROMPT

The canonical evaluation template appears below; every target and non-contract comment is stripped before insertion, and general annotation rules are supplied by the common system prompt. Both the SFT and RL data use this single prompt pair, and no model-visible prompt contains a target clause or non-contract comment.

Generation prompt

```
You are given a C function. Produce the strongest inductive ACSL loop
invariants that capture the loop's behavior (conservation/relational laws
+ tight progress bounds). Output ONLY invariant lines, each formatted
exactly as:
loop invariant <expr>;
Output at most 20 invariant lines.

Program:
{program}
```

The accompanying system prompt appears verbatim below.

System prompt

```
You are a formal verification expert specializing in ACSL loop invariant
synthesis for Frama-C/WP.

## LOOP INVARIANT DEFINITION

A loop invariant is a property that:
1. Holds before the first loop iteration (initialization).
2. Is preserved by every loop iteration when the loop guard is true
(inductiveness).
3. Combined with loop exit ('!guard'), helps establish required
loop-exit facts.

## RULES

- Add one core conservation/relational equality that explains loop
updates (state transition law).
- Add minimal progress bounds for loop counters and key arithmetic
variables.
- Prefer strong invariants; avoid weak/tautological ones.
- Do not add unrelated invariants just to increase count.
- MUST NOT modify or rewrite any 'unknown()'/'unknownN()' call.
- '\at(v, Pre)' can ONLY be used with function parameters, never with
local variables initialized inside the function.
- '\at(v, LoopEntry)' denotes the value of v at the start of the loop
and can ONLY be used with local variables initialized before the
loop (using v = unknown();).
- Loop guard is NOT an invariant; weaken strict guards to non-strict
bounds at exit.
- Parenthesize mixed equality and relational expressions so their
grouping is explicit.
- Use 'loop invariant' only as a line prefix, never inside expressions.
- Emit scalar invariant expressions only; do not declare predicates,
logic functions, axioms, or lemmas.
- Do NOT use the ternary '? :' operator; split cases with '==>' instead.
- '^' is bitwise XOR in C/ACSL, not exponentiation --- write 'x*x*x'
for x cubed.
- Do NOT place a C boolean directly in arithmetic ('x + (a > b)' is
invalid); use '==>' to split cases.
- Only use variables declared in the given function.
- Operators: comparison '==' '!=' '<' '<=' '>' '>='; logical '&&' '||'
'!' '==>' '<==>'; arithmetic '+' '-' '*' '/' '%'.

## ACSL INVARIANT SYNTAX
```

```

1242
1243 Allowed expression forms (all verified by Frama-C/WP):
1244
1245 1. **Bounds**: '0 <= i', 'i <= n', 'x >= 1'
1246 2. **Linear equality**: 'i + c == n', '2 * s == i * (i - 1)',
1247   'x == q * y + r'
1248 3. **Polynomial equality**: 'x == n * n * n',
1249   '6 * s == i * (i + 1) * (2 * i + 1)'
1250 4. **Relational identity** (multi-var algebraic law):
1251   '1 + x * z - x - z * y == 0', 'p * s - r * q == 1'
1252 5. **Implication**: 'i > 0 ==> s >= i', 'a != 0 ==> q >= 1'
1253 6. **Modular**: 'i % 5 == 0', 'r >= 0 && r < y'
1254 7. **Bitshift** (power-of-2 only): 'p == 1 << i', 's == (1 << i) - 1'
1255 8. **Conjunction**: 'c >= 1 && c <= k'
1256 9. **\at for params**: 'n == \at(n, Pre)' (function parameters only)
1257
1258 Forbidden:
1259 - `` for exponentiation (it is XOR) --- use repeated '*' or '<<' for
1260   powers of 2.
1261 - '\product', '\sum', '\factorial' --- not ACSL scalar operators.
1262 - '? :' ternary --- rewrite with '==>'.
1263 - '\old(x)' --- use '\at(x, Pre)'.
1264 - Type casts '(int)', '(long)' --- unnecessary in ACSL logic.
1265 - Function calls, predicates, lemmas, axioms --- emit only scalar
1266   expressions.
1267
1268 Operators (complete list):
1269   comparison: '==', '!=', '<', '<=', '>', '>='
1270   logical: '&&', '||', '!', '==>', '<==>'
1271   arithmetic: '+', '-', '*', '/', '%', '<<', '>>'
1272
1273 ## OUTPUT
1274
1275 Return ONLY invariant lines, one per line, formatted exactly as:
1276 'loop invariant <property>;'
1277 Do not return code fences, 'loop assigns', the surrounding C function,
1278 or explanations.
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295

```

F TRAINING INTERFACE

The training service is packaged as a self-contained, CPU-only container with gcc, the verifier, Why3, and its prover backend. It accepts program source, an array of model responses, one of the four reward variants defined below, and either random or structured negative sampling. The reward and sampler choices are independent; the full compositional reward and structured sampler are the defaults. The response reports each rollout's reward together with its coverage and Shapley-credit components. Responses may be raw LLM text, invariant lists, or annotated code.

G TRAINING-EVALUATION OVERLAP AUDIT

We compare the final RL (4,992 records and 4,992 distinct program sources) and SFT (33,346 records and 31,865 distinct program sources) training sets against all 832 evaluation programs after target removal. Under three increasingly aggressive normalizations—exact text, alpha-renamed identifiers, and alpha-renaming with integer constants abstracted—no training program matches an evaluation program (Table 4).

We separately measure coarse structural overlap. A structural cell combines the control-flow profile, guard kind, nonlinearity, nondeterminism, and a variable-count band. The evaluation set occupies 122 such cells. RL and SFT share 88 and 41 of them, respectively; their union shares 92 cells. Equivalently, 702 (84.4%), 529 (63.6%), and 754 (90.6%) of the 832 evaluation programs fall in a cell represented by RL, SFT, and their union. The record-weighted total-variation distances over cells are 0.422 for RL, 0.651 for SFT, and 0.596 for their union, so the training and evaluation distributions are related but not identical. Table 5 gives interpretable feature marginals behind this joint-cell comparison.

	RL	SFT	RL $\cup$ SFT
Records	4,992	33,346	38,338
Distinct program sources	4,992	31,865	35,170
Exact-text matches	0	0	0
Alpha-renamed matches	0	0	0
Alpha-renamed, constant-abstracted matches	0	0	0
Shared evaluation cells	88 / 122	41 / 122	92 / 122
Evaluation programs in shared cells	702 (84.4%)	529 (63.6%)	754 (90.6%)
Training records in evaluation-supported cells	4,992 (100%)	27,275 (81.8%)	32,267 (84.2%)
TV distance over structural cells	0.422	0.651	0.596

Table 4: Instance and structural-distribution audit of the final training sets against the 832 target-removed evaluation programs. Distribution statistics count training records, matching their sampling frequency.

Feature (% of records/programs)	Evaluation	RL	SFT
Constant guard	24.6	7.5	4.6
Single-variable guard	31.4	31.6	44.0
Variable-vs.-variable guard	37.3	57.3	50.6
Compound guard	6.7	3.7	0.8
Nonlinear	2.5	2.5	10.8
Nondeterministic	28.6	30.3	14.9
≤ 2 variables	35.1	21.2	46.2
3–4 variables	33.9	31.7	12.0
≥ 5 variables	31.0	47.1	41.8

Table 5: Marginal structural distributions of evaluation programs and final training records. The structural-cell TV distances in Table 4 use the joint features rather than these marginals independently.

H TRAINING-DATA CURATION

The released RL and SFT training sets use the target-hidden scalar-integer single-loop interface. SFT targets are inductive invariant sets synthesized by the multi-round procedure in Section 4.3 and accepted by the loop verifier. The only learned generator is the open-weight model being trained; target acceptance and RL reward computation use program execution and verification rather than labels or critiques from an external model. Table 6 summarizes the released sets.

RL set	4,992 records over 1,131 distinct loop shapes
SFT set	33,346 records over 31,865 distinct program sources
RL–SFT exact-source overlap	1,687 distinct programs

Table 6: Released training sets.

The reward discriminates on the released data. On the 125 released RL programs with a reference answer, the production reward gives the reference a median of 0.845 and its bounds-only projection a median of 0.0; the tautology $1 = 1$, the loop guard copied as an invariant, and an unsound clause score exactly zero on every program. A simulated eight-answer group has a median within-group reward standard deviation of 0.21, so group-relative updates retain a usable advantage signal.

I EXPERIMENTAL PROTOCOL DETAILS

The evaluation workload composition is given in Section 5.1; the 466 Loopy programs are the integer subset of its original 469-program corpus (three floating-point programs excluded), and unsupported inputs remain failures in the common denominator.

Before model invocation, we remove assert and ensures predicates, executable assertion calls, conventional error-label names, and non-contract comments, including source-provenance paths; preconditions and executable semantics remain visible. CRAFT evaluation runs use the canonical generation template, extractor, canonical-key deduplication, and Houdini implementation; external tools retain their own orchestration. Each model retains its serving interface and chat template. All API and locally served CRAFT checkpoints use non-reasoning decoding. Decoding uses temperature ϑ , nucleus parameter p , and completion caps L_{api} and L_{local} for API and local models, respectively, without an additional top- k , repetition penalty, re-roll, or target feedback. Each RQ1 curve reuses the same saved response pool per task. The model-specific pool sizes and the common reported budget grid are listed in Table 3; every pool contains at least k_{max} responses. For a pool with c standalone successes among n responses, $\text{pass}@k$ uses $1 - \binom{n-c}{k} / \binom{n}{k}$; $\text{compose}@k$ filters the first k responses in the saved order.

Training configuration. Tables 7 and 8 record the supplied SFT and RL run configurations. All reported SFT checkpoints are trained for three epochs, and all reported RL checkpoints are trained for five epochs. Reward and sampler constants are reported separately in Table 3.

Field	Value
Learning rate η_{sft}	1×10^{-5}
Epochs e_{sft}	3
Batch (per device $\times$ accumulation $\times$ GPUs)	global batch 32; $2 \times 2 \times 8$ or $1 \times 4 \times 8$
Sequence cutoff L_{sft}	2,048
DeepSpeed	ZeRO-3
Scheduler	cosine

Table 7: SFT training configurations.

1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457

Field	Value
Batch size b_{rl}	64
Learning rate η_{rl}	1×10^{-6}
KL coefficient β	0.001
Advantage stabilizer δ_{adv}	10^{-6}
Rollouts per prompt g_{rl}	8
Tensor parallelism p_{tp}	1–2
GPU memory utilization μ_{gpu}	0.3–0.5
Epochs e_{rl}	5
Total steps s_{rl}	200
Checkpoint interval s_{ckpt}	100

Table 8: RL training configurations.

J COMPLETE EXPERIMENTAL DATA

This section collects the per-model and per-stratum measurements behind the main aggregates.

J.1 PROBE CURVES

Table 9: Complete whole-workload Bare probe curves (% verified). k_α is the smallest measured budget at which $\text{compose}@k$ reaches fraction α of $\text{compose}@k_{\max}$.

Model	Metric	1	2	4	8	16	32	k_α
Qwen3-1.7B	pass	7.74	11.22	15.02	18.77	22.41	25.99	32
	compose	19.47	23.68	28.97	33.05	35.34	37.86	
Qwen3-4B	pass	6.42	9.02	11.66	14.05	16.49	19.12	16
	compose	32.93	38.94	44.11	47.60	50.72	52.76	
Qwen3-8B	pass	6.43	9.54	13.24	17.33	21.62	25.96	16
	compose	37.86	45.07	50.60	54.21	56.37	57.81	
Qwen3-14B	pass	7.67	10.74	14.17	17.97	22.04	26.13	8
	compose	43.51	51.20	58.29	61.54	63.70	64.54	
Qwen3-30B-A3B	pass	1.40	2.57	4.53	7.65	12.15	17.53	16
	compose	17.43	23.92	30.65	36.54	42.31	43.87	
Llama 3.1 8B	pass	0.61	1.16	2.16	3.81	6.30	9.76	8
	compose	27.88	37.14	46.03	51.20	51.68	52.16	

Table 10: Per-GPU decoding throughput of the four backbones used in training experiments, measured with vLLM on A800-80G GPUs using response budget n_{probe} , temperature ϑ , and completion cap L_{local} .

Model	tok/s/GPU
Qwen3-4B	7,445
Qwen3-8B	5,833
Qwen3-14B	3,168
Llama 3.1 8B	5,135

J.2 SFT PROBE CURVES

The SFT checkpoints use rejection-filtered synthetic targets and are evaluated under the same 832-task protocol as the base probe, using response budget n_{probe} , temperature ϑ , per-obligation prover budget δ , and the current prompt. All three checkpoints are evaluated after three epochs and use the SFT synthesis and verification protocol defined in Section 4.3.

Table 11: Complete SFT-checkpoint probe results by benchmark stratum and sampling budget. Each cell reports a percentage; All / 832 is the whole-workload result. k_α is the smallest measured budget at which $\text{compose}@k$ reaches fraction α of $\text{compose}@k_{\max}$. Bold marks the best value in each column at a fixed budget.

Model	k	Linear / 316		NLA / 50		Loopy / 466		All / 832		k_α
		pass	comp	pass	comp	pass	comp	pass	comp	
Qwen3-4B + SFT	1	43.69	65.82	2.08	8.00	42.34	66.31	40.43	62.62	8
	4	58.55	78.48	4.81	10.00	58.38	76.82	55.23	73.44	
	8	63.36	81.01	6.23	12.00	63.47	79.83	59.99	76.20	
	16	67.55	82.28	7.99	12.00	67.77	80.90	64.09	77.28	
	32	71.22	82.28	10.36	12.00	71.54	81.97	67.74	77.88	
Qwen3-8B + SFT	1	45.69	71.52	3.60	8.00	43.50	67.60	41.93	65.50	4
	4	60.99	79.43	6.26	12.00	59.41	78.97	56.82	75.12	
	8	65.36	80.70	7.34	14.00	64.00	82.19	61.11	77.52	
	16	69.01	81.33	8.44	14.00	67.78	83.69	64.68	78.61	
	32	72.34	81.96	9.64	14.00	71.28	83.91	67.98	78.97	
Qwen3-14B + SFT	1	50.18	70.25	3.54	12.00	47.12	69.53	45.66	66.35	4
	4	63.93	81.33	7.27	16.00	61.46	80.04	59.14	76.68	
	8	68.00	82.28	9.79	16.00	66.21	82.62	63.50	78.49	
	16	71.42	82.91	12.06	16.00	70.50	83.69	67.34	79.33	
	32	74.17	82.91	13.52	16.00	74.25	83.91	70.57	79.45	

Table 12: Per-GPU decoding throughput of the SFT checkpoints, measured with vLLM on A800-80G GPUs using 100 responses per task, temperature ϑ , and completion cap L_{local} .

Model	tok/s/GPU
Qwen3-4B + SFT	7,445
Qwen3-8B + SFT	5,833
Qwen3-14B + SFT	3,168
Llama 3.1 8B + SFT	5,135

J.3 REINFORCEMENT-LEARNING RESULTS

Table 13: Complete RL-from-Zero comparison with the SFT checkpoint shown as a separate reference. All rows use the same decoding and verification configuration. Unmarked rows are Qwen3-8B.

Checkpoint	Stage	pass@1	compose@1	compose@8	compose@16	compose@32	k_α
Zero	before RL	6.43%	37.86%	54.21%	56.37%	57.81%	16
RL-Zero	after RL	6.80%	57.93%	70.43%	72.60%	73.20%	8
SFT	reference	41.93%	65.50%	77.52%	78.61%	78.97%	4
Zero (Qwen3-14B)	before RL	10.1%	52.5%	65.0%	66.2%	67.8%	8
RL-Zero (Qwen3-14B)	after RL	35.60%	70.55%	77.04%	77.64%	78.00%	4

J.4 ABLATION RESULTS

The reward variants correspond directly to the two levels of Equation 6. *Binary Inductiveness* assigns one iff the processed response is nonempty and every clause survives Houdini. *Whole-Rollout Strength* pays the strength score $\text{cov}(V_i)$ only when every processed clause survives,

$$r_i^{\text{whole}} = \mathbf{1}[V_i = \text{Unique}(\text{First}_{m_{\max}}(\text{parts}(y_i)))] \frac{|R_i|}{|\mathcal{N}|}.$$

Clause-Decomposed Strength removes this all-or-nothing indicator and scores each response by the coverage of its surviving V_i ; the *Full Compositional Credit* additionally adds the group credit $w_g \phi_i$. All four variants admit at most $m_{\max}$ clauses per response. We refer to the four variants as Binary, Whole-rollout, Clause-decomposed, and Full in figures and prose. All variants train from the Zero initialization on the curated pool with identical optimization and inference settings.

Table 14: Reward-function ablation on Qwen3-8B, trained from the Zero initialization (%); exact numbers for Figure 4. Best result per column among trained variants in bold.

Variant	pass@k					compose@k				
	1	4	8	16	32	1	4	8	16	32
Zero (untrained reference)	6.43	13.24	17.33	21.62	25.96	37.86	50.60	54.21	56.37	57.81
Binary Inductiveness	3.78	6.21	7.72	9.40	11.26	7.21	12.26	16.71	20.31	23.32
Whole-Rollout Strength	16.67	19.17	20.03	20.90	21.92	18.87	19.95	20.31	21.27	22.72
Clause-Decomposed Strength	4.10	9.35	12.27	14.88	17.14	48.08	57.33	60.22	61.54	61.90
Full Compositional Credit (ours)	6.80	13.41	16.80	20.17	23.44	57.93	66.95	70.43	72.60	73.20

The negative-sampler ablation uses matched Qwen3-8B runs from the Zero initialization under the full compositional reward that differ only in the sampler. Structured is the sampler of Section 4.2 (relation and post-exit families); Random draws uniform local perturbations under the same trace budget.

Table 15: Negative-sampler ablation on Qwen3-8B, trained from the Zero initialization under the full reward (%). Bold marks the better sampler per column.

Negative sampler	pass@ k					compose@ k				
	1	4	8	16	32	1	4	8	16	32
Random	7.19	11.78	13.86	15.89	17.98	46.88	55.41	58.77	60.82	62.26
Structured (ours)	7.12	12.81	15.63	18.34	20.75	48.92	59.62	63.34	66.11	67.31

Structured sampling improves compose@ k by 2.0–5.3 points at every budget, whereas pass@ k stays within -0.1 to 2.8 points of Random.

J.5 PREDICTIVE VALIDITY OF NEGATIVE COVERAGE

We evaluate the target-independent strength signal on 6,190 saved candidate invariant sets produced by eight target-hidden generation pipelines on the 832-program test set. For each candidate, the prediction score is its structured relation/post-exit negative coverage, computed without $\mathcal{G}$; the binary label is the final verifier verdict after the untouched target is restored. Predictors use coverage alone; restored targets supply evaluation labels.

The continuous analysis contains 5,606 candidates from 752 programs. The remaining 584 candidate rows belong to 80 programs for which the sampler retains no negative group; those rows use the binary reward fallback and are excluded from the continuous-coverage analysis. Because multiple candidate generators are evaluated on the same program, confidence intervals use 5,000 stratified program-cluster bootstrap replicates: all candidates for a resampled program move together. A second analysis computes AUROC separately within each of the 547 programs having both successful and unsuccessful candidates, then macro-averages over programs. This removes between-program difficulty as an explanation for the ranking.

Population	Candidates	Positive rate	AUROC	AUPRC
All	5,606	0.508	0.738	0.730
Linear	2,336	0.533	0.743	0.768
NLA	400	0.090	0.542	0.132
Loopy	2,870	0.547	0.764	0.778

The pooled AUROC has a program-clustered 95% confidence interval of $[0.711, 0.763]$, and the AUPRC interval is $[0.699, 0.758]$, compared with the pooled positive-rate baseline of 0.508. Within programs, macro AUROC is 0.794, with a clustered interval of $[0.773, 0.814]$; randomly permuting target verdicts among candidates for the same program gives a one-sided $p < 2 \times 10^{-4}$ over 5,000 permutations. Method-stratified AUROCs range from 0.572 to 0.731, so the pooled association is not created by a single candidate generator. Candidates with coverage at least 0.90 verify 78.9% of the time, compared with 29.2% below 0.50.

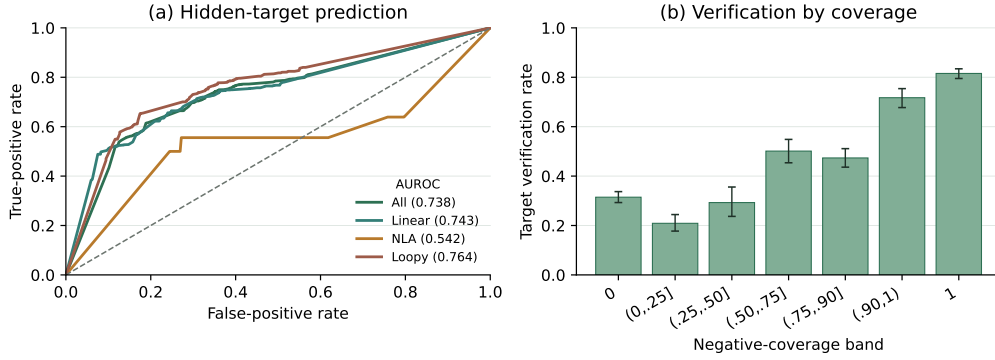


Figure 6: Predictive validity of target-independent negative coverage on saved test-set candidates. (a) ROC curves for restored-target verification; numbers in parentheses are AUROCs. (b) Target verification rate by coverage band, with Wilson 95% confidence intervals. NLA remains close to chance, showing that the current structured negatives discriminate hidden-target utility much better for Linear and Loopy programs than for nonlinear arithmetic.

Coverage ranks candidate strength. The NLA stratum identifies where richer nonlinear perturbations are needed.

J.6 CROSS-MODEL MAIN RESULTS

Table 16: Complete reasoning-off cross-model results by benchmark stratum and sampling budget. Each cell reports a percentage; All / 832 is the whole-workload result. Rows for each model come from one archived rollout pool of 10 responses per task. $\text{compose}@k$ uses prefix composition and $\text{pass}@k$ uses the unbiased estimator. Bold marks the best value in each column at a fixed budget. GPT-5-mini is excluded because it has no reasoning-off setting.

Model	k	Linear / 316		NLA / 50		Loopy / 466		All / 832	
		pass	comp	pass	comp	pass	comp	pass	comp
GPT-5-nano	1	3.77	34.81	0.40	22.00	3.80	34.55	3.58	33.89
	4	10.79	48.10	1.60	30.00	11.48	51.72	10.62	49.04
	8	15.84	55.38	3.20	44.00	17.58	56.87	16.05	55.53
GPT-5	1	35.06	52.22	5.80	36.00	33.78	52.36	32.58	51.32
	4	47.73	64.87	8.74	52.00	50.67	67.38	47.03	65.50
	8	51.93	70.25	9.60	66.00	56.13	73.82	51.74	72.00
Claude Sonnet-4.6	1	34.49	55.38	7.80	44.00	40.67	59.44	36.35	56.97
	4	40.32	62.97	8.80	52.00	47.27	66.31	42.32	64.18
	8	42.87	66.46	9.60	64.00	50.12	67.60	44.93	66.95
DeepSeek V4-Flash	1	28.67	52.22	5.00	44.00	30.15	51.72	28.08	51.44
	4	42.35	66.46	7.52	60.00	47.73	69.96	43.27	68.03
	8	46.64	71.52	8.00	72.00	53.59	76.61	48.21	74.40

$\text{Pass}@k$ and $\text{compose}@k$ use the same archived response pool for each model. $\text{Pass}@k$ is the unbiased pool estimator, whereas $\text{compose}@k$ evaluates the first k saved responses.

J.7 COMPARISON WITH SPECIALIZED TOOLS

Table 17: Archived target-hidden tool comparison: the non-reasoning rows from the main text alongside the archived medium-reasoning rerun; Daikon makes no model call and is listed once. Each cell reports the verification rate for Linear, NLA, Loopy, or all 832 programs. Medium-effort results are available only for the single-response measurements. The non-reasoning compose@1, compose@4, and compose@8 rows are prefix compositions of the same archived ten-rollout pool, as in Table 2.

Method	Reason.	Linear	NLA	Loopy	All	Tokens/task	Time/task
AutoSpec	none	47.78%	6.00%	42.27%	42.19%	2,181.72	27.34 s
AutoSpec	medium	65.82%	8.00%	64.81%	61.78%	25,154.54	54.77 s
SESpec	none	28.80%	4.00%	33.05%	29.69%	22,081.61	68.50 s
SESpec	medium	56.96%	6.00%	49.57%	49.76%	44,807.05	117.31 s
Clause2Inv	none	20.89%	0.00%	0.00%	7.93%	1,056.25	8.41 s
Clause2Inv	medium	19.94%	2.00%	0.00%	7.69%	385.80	29.04 s
Daikon	—	23.10%	0.00%	21.67%	20.91%	0	4.89 s
Loopy	none	20.25%	0.00%	19.74%	18.75%	14,513.37	147.16 s
Loopy	medium	72.78%	12.00%	70.82%	68.03%	111,973.15	381.01 s
Naive	none	15.19%	2.00%	18.88%	16.47%	225.95	2.98 s
Naive	medium	48.73%	8.00%	47.42%	45.55%	4,493.37	28.87 s
CRAFT (pass@1)	none	2.53%	0.00%	4.72%	3.61%	1,135.83	7.86 s
CRAFT (pass@1)	medium	61.08%	14.00%	54.08%	54.33%	6,928.30	53.52 s
CRAFT (compose@1)	none	34.81%	22.00%	34.55%	33.89%	1,136.82	24.52 s
CRAFT (compose@1)	medium	64.24%	14.00%	63.09%	60.58%	6,928.30	66.60 s
CRAFT (compose@4)	none	48.10%	30.00%	51.72%	49.04%	1,905.10	46.40 s
CRAFT (compose@8)	none	55.38%	44.00%	56.87%	55.53%	2,929.47	69.99 s

The main text reports each method’s non-reasoning row. The shared model-call parameters for the non-reasoning comparison are summarized separately in Table 18.

Table 18: Sampling parameters used in the non-reasoning target-hidden tool comparison.

Method	Sampling budget / task	Temperature / top- p	Completion cap	Reasoning
AutoSpec	8	1.0 / 1.0	8,192	none
SESpec	1 initial + up to 3 refinements per phase	1.0 / 1.0	8,192	none
Clause2Inv	adaptive; 1 per call	1.0 / 1.0	8,192	none
Daikon	no model call	—	—	—
Naive	1	1.0 / 1.0	8,192	none
CRAFT	1, 4, or 8	1.0 / 1.0	8,192	none
Loopy	15 initial + up to 7 repairs	0.7 / 1.0	8,192	none

Token totals are means over tasks with at least one model call; accuracy always keeps the 832-task denominator.

J.8 TOOL-SPECIFIC BEHAVIOR UNDER TARGET HIDING

AutoSpec. AutoSpec pools eight parallel proposals in its native orchestration. The reported result is produced by this multi-proposal pipeline.

Clause2Inv. Clause2Inv normally constructs false-state counterexamples from the postcondition. Under target hiding, this source of counterexamples is unavailable. Its transition verification conditions are also unavailable for the 466 Loopy programs; these instances remain failures in the common denominator.

1836 **Loopy.** Loopy’s initial responses contain textual loop-invariant annotations in 98.8% of cases, and
1837 its native extractor passes 1.75% of initial responses to Houdini. These rates measure the interface
1838 between generated text and the clauses accepted by its downstream pipeline.
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889

K DETAILED LIMITATIONS

Program scope. Our implementation targets numerical programs with one scalar-integer loop. The evaluation excludes nested loops and invariants over arrays, heaps, or recursively defined predicates. The negative sampler covers scalar arithmetic relations and exit boundaries (Liu et al., 2024a).

Inference engine. Local base and trained checkpoints are served with vLLM. In preliminary comparisons under nominally matched decoding settings, these checkpoints had lower verification rates with vLLM than with the other evaluated backends. API models retain their provider serving interfaces and chat templates. The reported decoding and reasoning controls are applied to each backend.

Verifier and specification language. Our filtering and all final judgments use Frama-C/WP, and candidates use ACSL. Parser behavior, generated obligations, prover integration, and syntax affect which clauses survive. The reported results characterize this verifier, specification language, and source language.

Finite candidate-pool approximation. Before Houdini, the pipeline implementation applies a lightweight AST-based filter to parse, normalize, and deduplicate candidate clauses. Clauses outside its supported ACSL subset are omitted. Each response contributes at most $m_{\max}$ clauses before global pooling; later clauses in that response are truncated, while no separate cap is applied to the pooled union.

Feedback regime. The evaluation covers target-independent training and finite-budget composition without iterative verifier-feedback optimization. In pilot runs, models reached inductive but weak invariants: the invariants passed induction, while the hidden goals left the verifier without a target-directed signal for the missing behavioral strength. The marginal gain from refinement decreased as the initial rollout pool grew.

L WHY THINKING IS DISABLED FOR LOCAL MODELS

The non-reasoning API experiments disable reasoning on compatible endpoints; Claude Sonnet disables adaptive thinking with a zero thinking budget. Local checkpoints use non-reasoning decoding during synthetic-data construction, training rollouts, and evaluation.

Explicit chain-of-thought adds reasoning tokens to every rollout, with total generation scaling with the sampling budget. The required output is a short, machine-parseable set of ACSL clauses. These clauses admit clause-level filtering and cross-rollout composition: Houdini can remove a failing clause, retain its siblings, and merge survivors from different rollouts. Reasoning traces are generated and consumed as whole sequences and are not included in this clause-level composition operator.

Table 19: Pre-filtering and post-filtering for invariant verification on the 832-program benchmark (Qwen3-8B, saved prefixes up to 32 rollouts; compose@ k in %). CPU-h is the whole-workload total.

Method	Subset	@1	@4	@8	@16	@32	CPU-h
Post-filter (300 s)	All	40.4	52.8	56.0	58.1	58.7	113
	Loopy	43.3	56.9	60.9	63.1	63.7	
	Linear	41.1	53.8	56.3	57.9	58.5	
	NLA	8.0	8.0	8.0	12.0	12.0	
Post-filter (900 s)	All	40.4	53.4	58.4	62.5	64.2	127
	Loopy	43.3	57.7	63.3	68.7	70.2	
	Linear	41.1	54.1	59.2	61.4	63.3	
	NLA	8.0	8.0	8.0	12.0	14.0	
Pre-filter	All	40.4	52.8	57.9	62.1	64.4	1472
	Loopy	43.3	57.3	62.9	68.0	71.0	
	Linear	41.1	53.2	58.5	61.4	63.0	
	NLA	8.0	8.0	8.0	12.0	12.0	

M POOLED FILTERING AND PROVENANCE CREDIT

Equations (25)–(26) define the common order: merge the clauses, filter the union once, and assign each survivor back to its proposers. RL, SFT, and inference use this pooled order. The Shapley term allocates coverage over the fixed survivor set.

Runtime and signal measurements. On a fixed stratified sample of 30 archived tasks (10 Linear, 10 NLA, and 10 Loopy), with eight rollouts per task and per-obligation budget δ , pooled filtering used 1.0 filter calls per task, compared with 8.6 for independent filtering. Total scoring time was 655.60 versus 824.39 seconds, and median task time was 6.48 versus 16.82 seconds. Against restored-target per-rollout verdicts, AUROC was 0.726 for pooled filtering and 0.725 for independent filtering; average precision was 0.454 for both.

Pre-filtering and post-filtering. Pre-filtering processes each response separately and unions its survivors. Post-filtering processes the pooled clause union once. The pooled candidate set admits cross-response support before deletion (Proposition 4).

Table 19 reports both placements on the 832-program benchmark with Qwen3-8B, evaluating saved prefixes up to 32 rollouts. Under the 300-second cap, post-filtering reaches 58.7 at $k=32$, compared with 64.4 for pre-filtering. With the 900-second cap, the corresponding result is 64.2. Whole-workload costs are 113, 127, and 1,472 CPU-hours for short-cap post-filtering, long-cap post-filtering, and pre-filtering, respectively.